CS 225

Data Structures

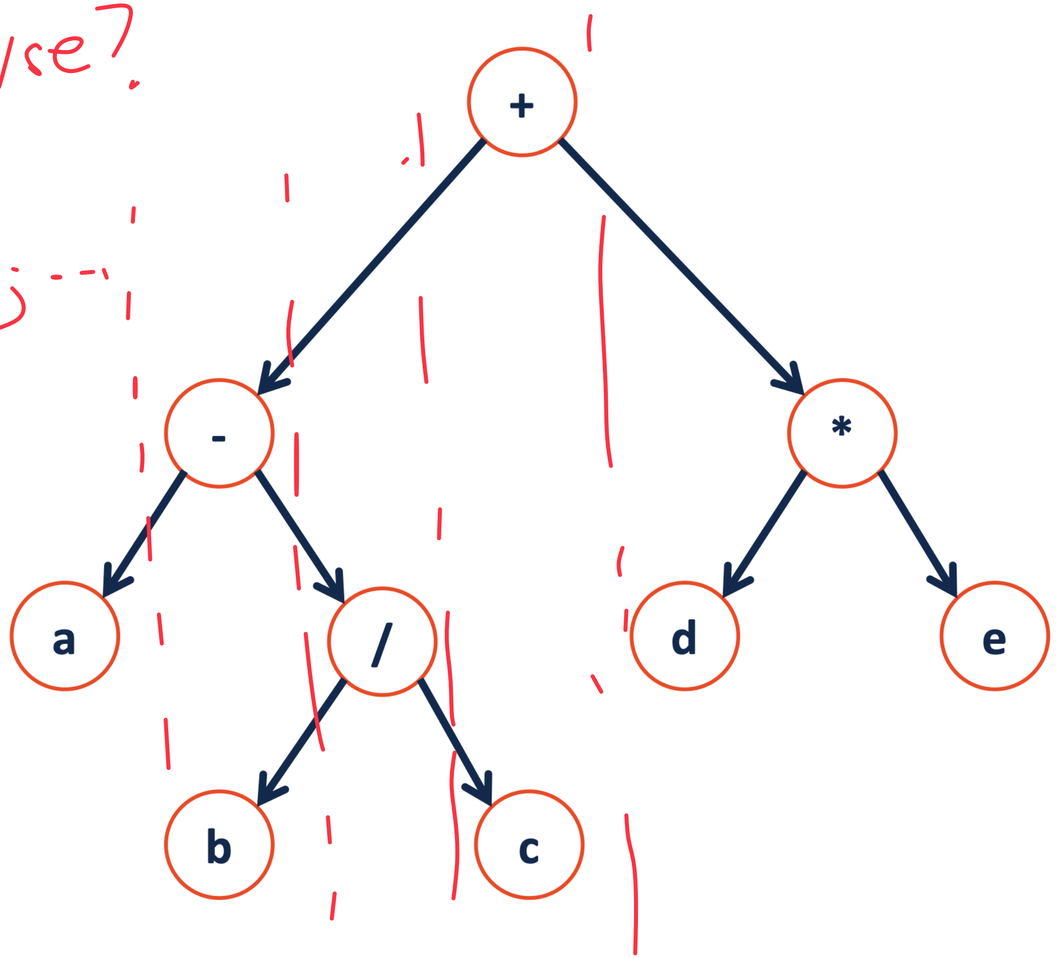
Volodymyr Kindratenko

Slides courtesy of Wade Fagen-Ulmschneider

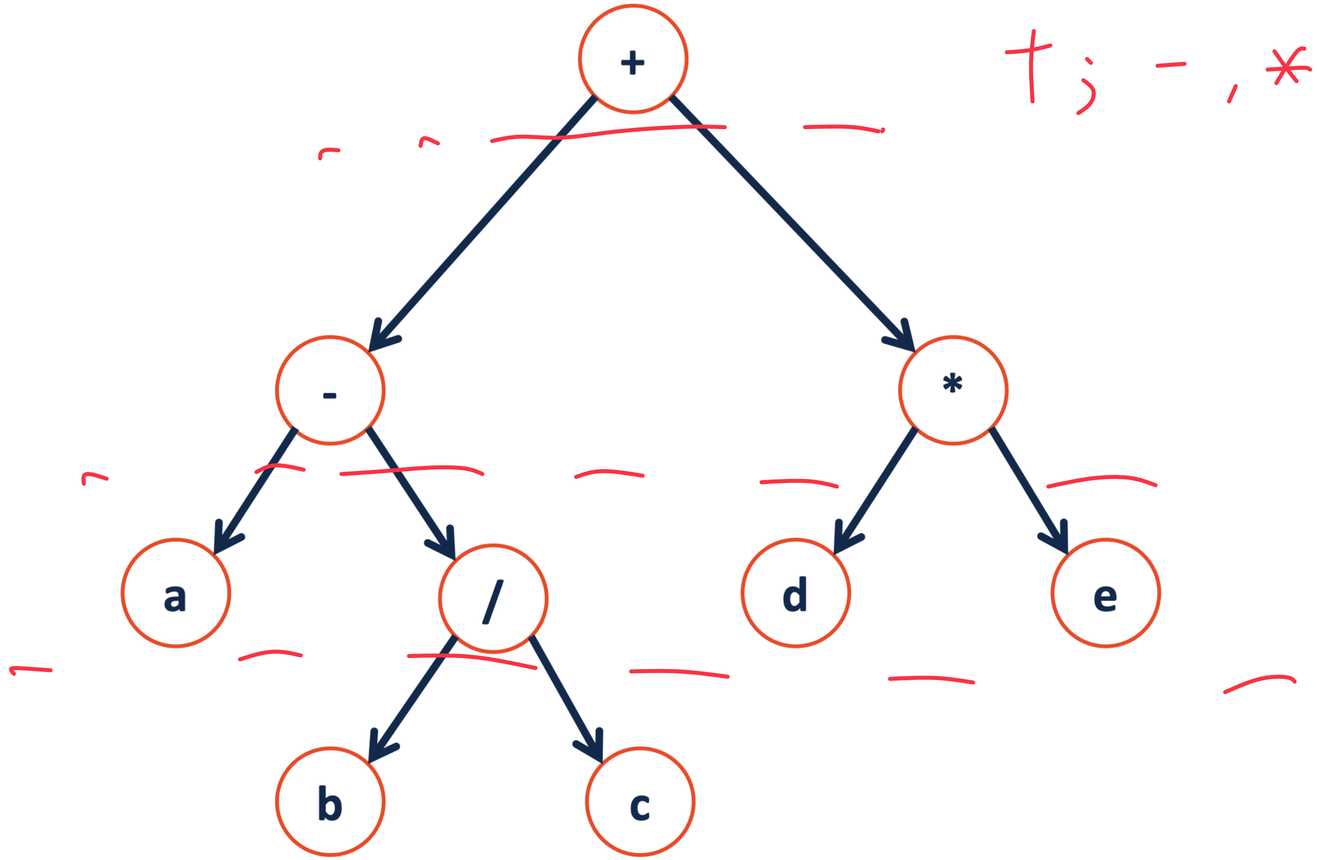
Traversals

How to traverse?

a; - , b; b; + , c; ---

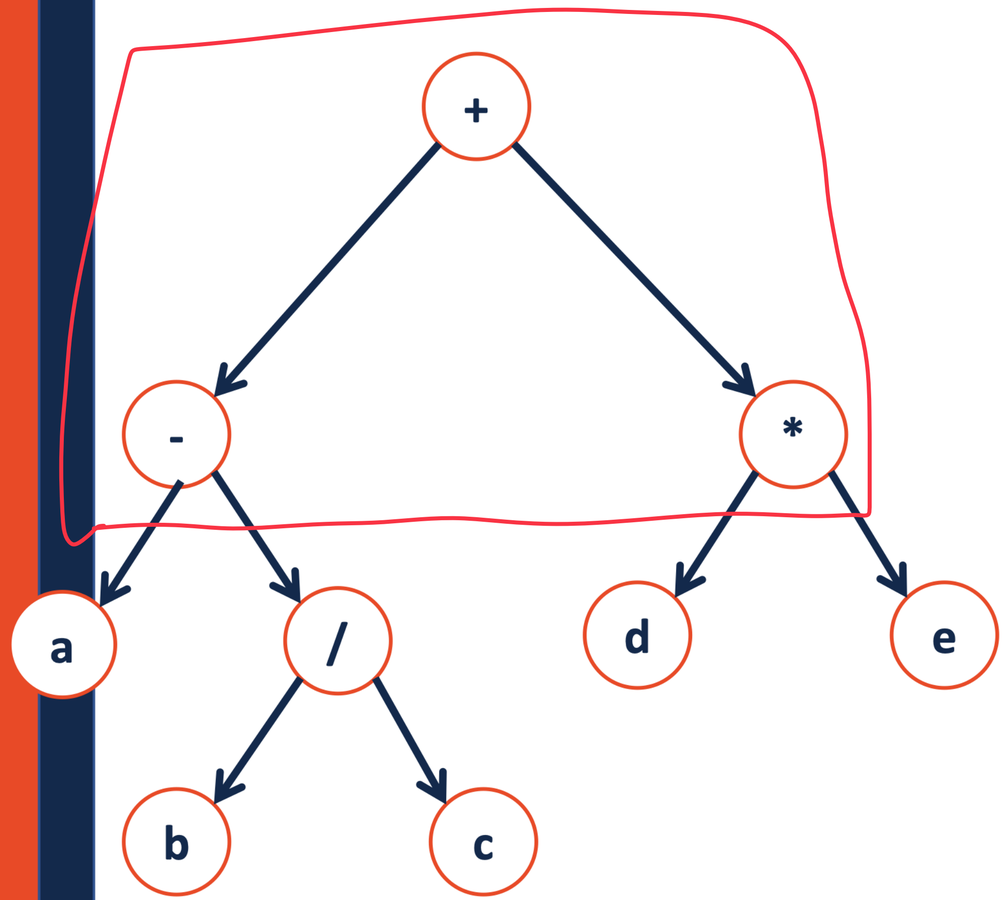


Traversals



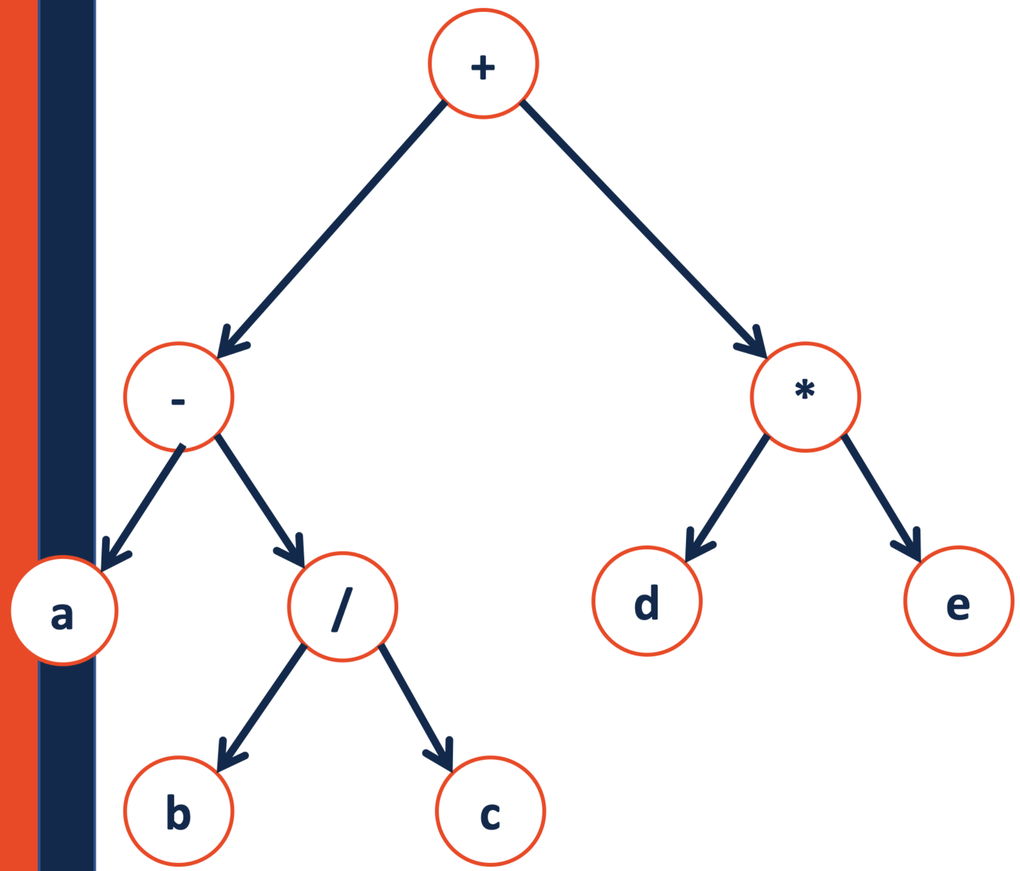
Traversals

- start with a small tree example
- do it recursively




```
1  template<class T>
2  void BinaryTree<T>::__Order(TreeNode * root)
3  {
4     if (root != NULL) {
5
6        ____________________________;
7
8        ___Order(root->left);
9
10       ____________________________;
11
12       ___Order(root->right);
13
14       ____________________________;
15
16    }
17 }
```

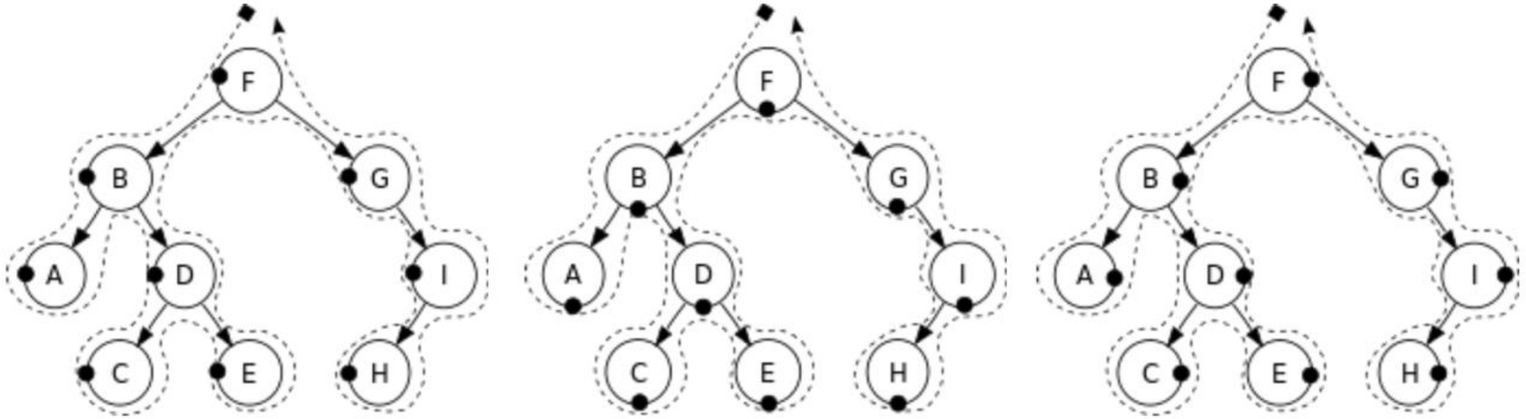
Traversals




```
1  template<class T>
2  void BinaryTree<T>::__Order(TreeNode * root)
3  {
4     if (root != NULL) {
5
6        ____________________________;
7
8        ___Order(root->left);
9
10       ____________________________;
11
12       ___Order(root->right);
13
14       ____________________________;
15
16    }
17 }
```

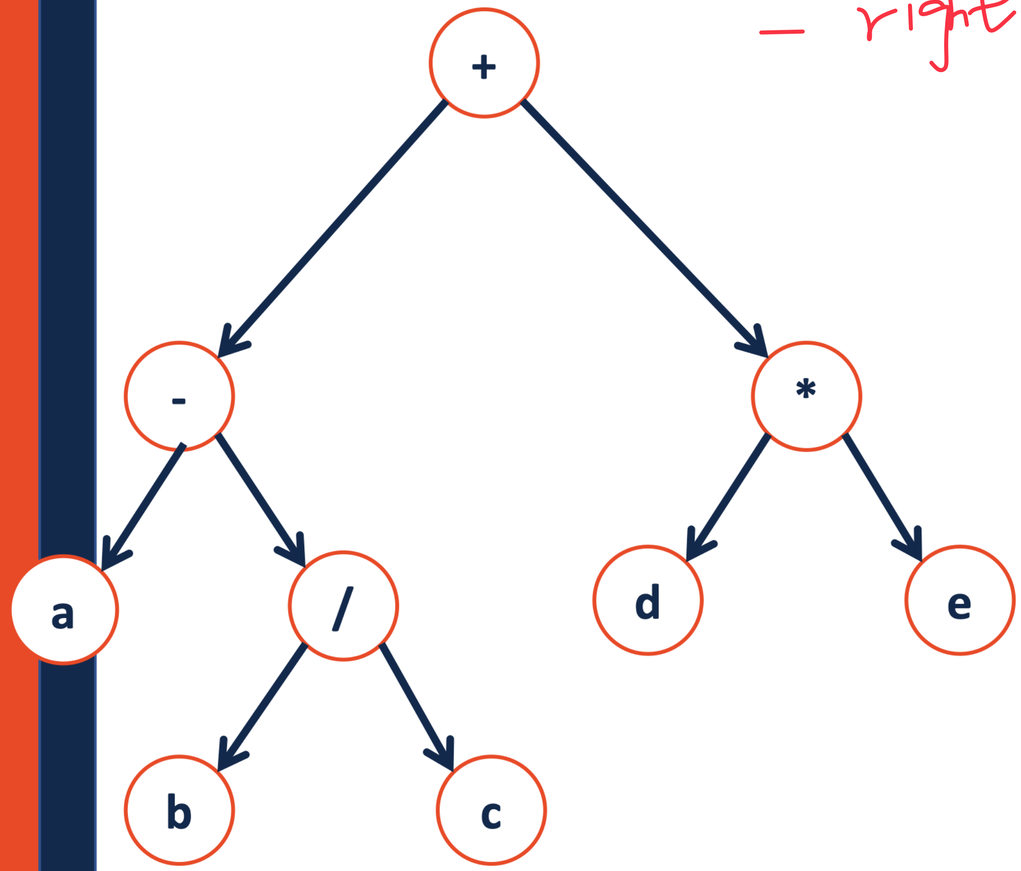
Tree traversals

- Depth-first: pre-order, in-order, post-order



Traversals

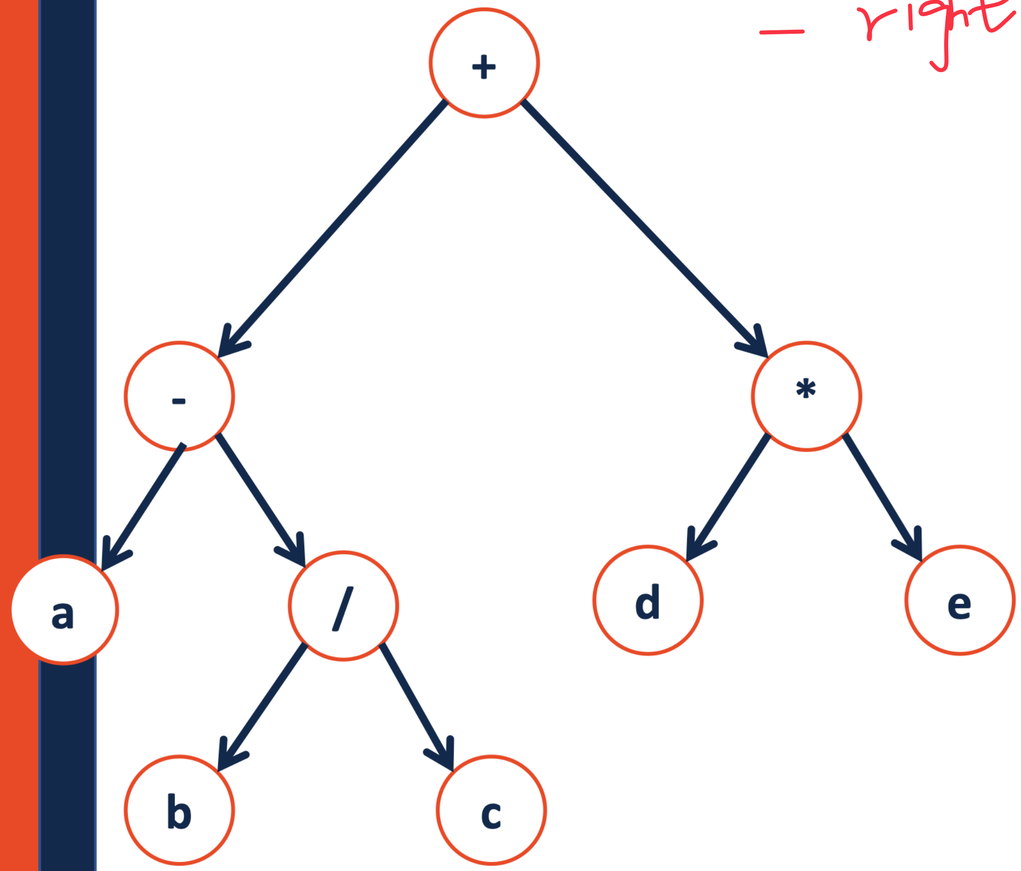
- left subtree
- root
- right subtree




```
1  template<class T>
2  void BinaryTree<T>::__Order(TreeNode * root)
3  {
4     if (root != NULL) {
5
6        ____________________________;
7
8        ___Order(root->left);
9
10       ____________________________;
11
12       ___Order(root->right);
13
14       ____________________________;
15
16    }
17 }
```

Traversals

- left subtree
- root
- right subtree

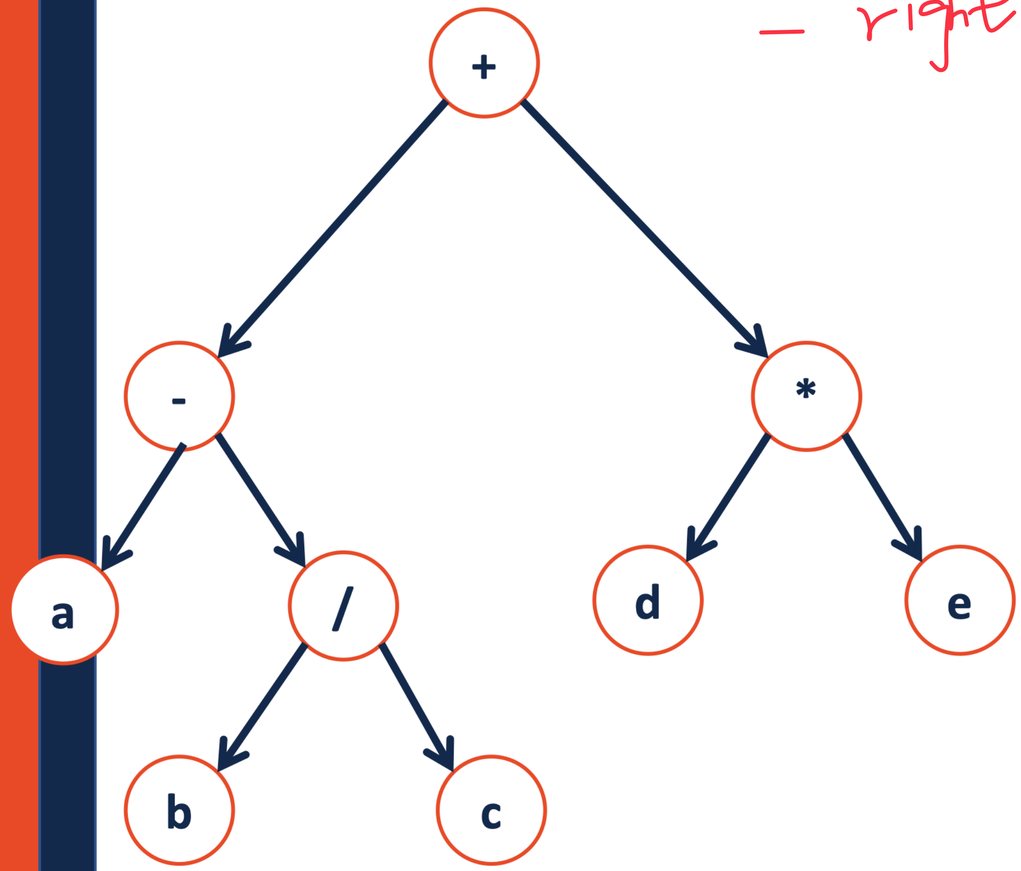



```
1  template<class T>
2  void BinaryTree<T>::In_Order(TreeNode * root)
3  {
4     if (root != NULL) {
5
6        _____________________________;
7
8        In_Order(root->left);
9
10       _____________________________;
11
12       In_Order(root->right);
13
14       _____________________________;
15
16    }
17 }
```

↳ a, -, b, /, c, +, d, *, e

Traversals

- left subtree
- root
- right subtree

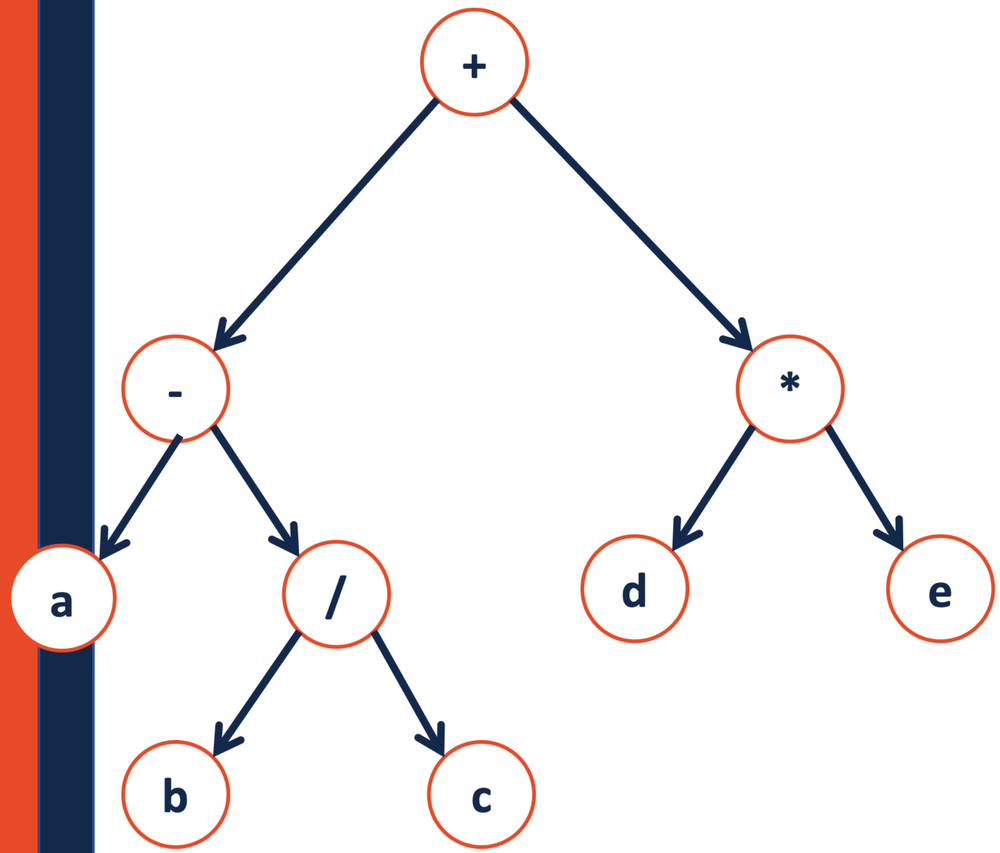



```
1  template<class T>
2  void BinaryTree<T>::__Order(TreeNode * root)
3  {
4     if (root != NULL) {
5
6        ____________________________;
7
8        __Order(root->left);
9
10       ____print(root);____;
11
12       __Order(root->right);
13
14       ____________________________;
15
16    }
17 }
```

↳ a, -, b, /, c, +, d, *, e

L, Self, R.

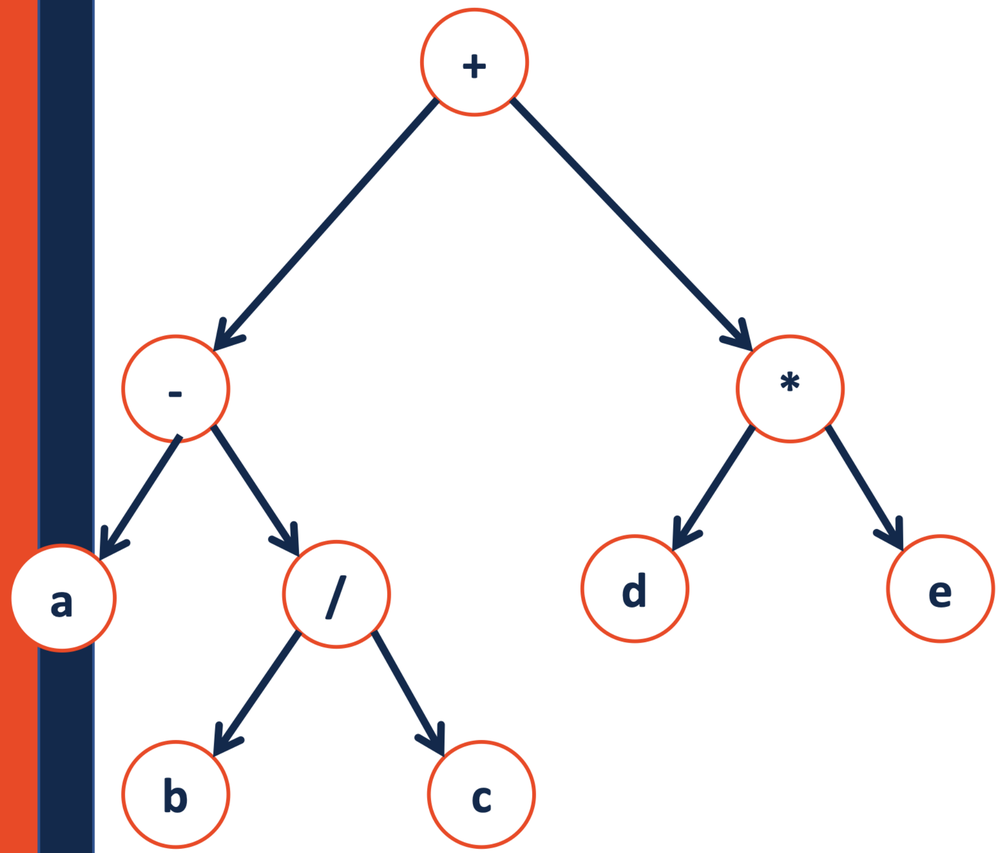
Traversals




```
1  template<class T>
2  void BinaryTree<T>::postOrder(TreeNode * root)
3  {
4     if (root != NULL) {
5
6        _____________________________;
7
8      postOrder(root->left);
9
10       _____________________________;
11
12     postOrder(root->right);
13
14       print ( root )              ;
15
16    }
17 }
```

L, R, Self

Traversals

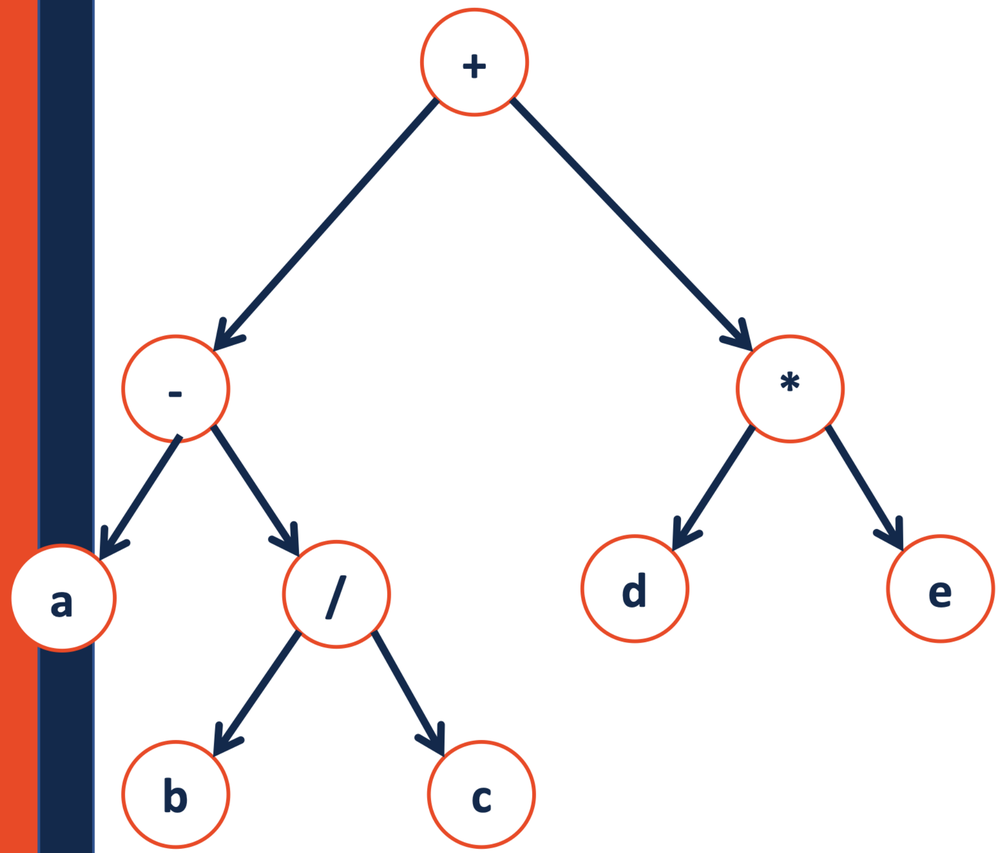


a, b, c, /, -, d, e, *, +

```
1  template<class T>
2  void BinaryTree<T>::postOrder(TreeNode * root)
3  {
4     if (root != NULL) {
5
6        _____;
7
8        postOrder(root->left);
9
10       _____;
11
12       postOrder(root->right);
13
14       print( root )      ;
15
16    }
17 }
```

L, R, Self

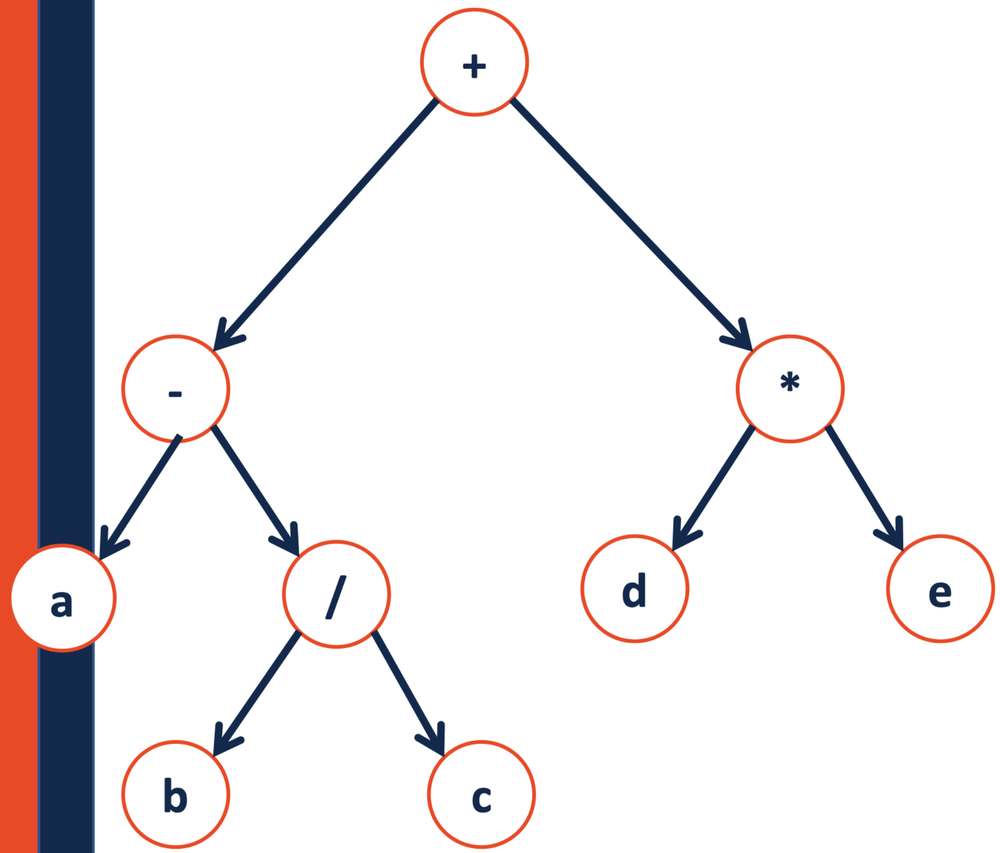
Traversals




```
1  template<class T>
2  void BinaryTree<T>::preOrder(TreeNode * root)
3  {
4     if (root != NULL) {
5
6           print(root)          ;
7
8        preOrder(root->left);
9
10          ____________________;
11
12       preOrder(root->right);
13
14          ____________________;
15
16    }
17 }
```

Self, L, R

Traversals

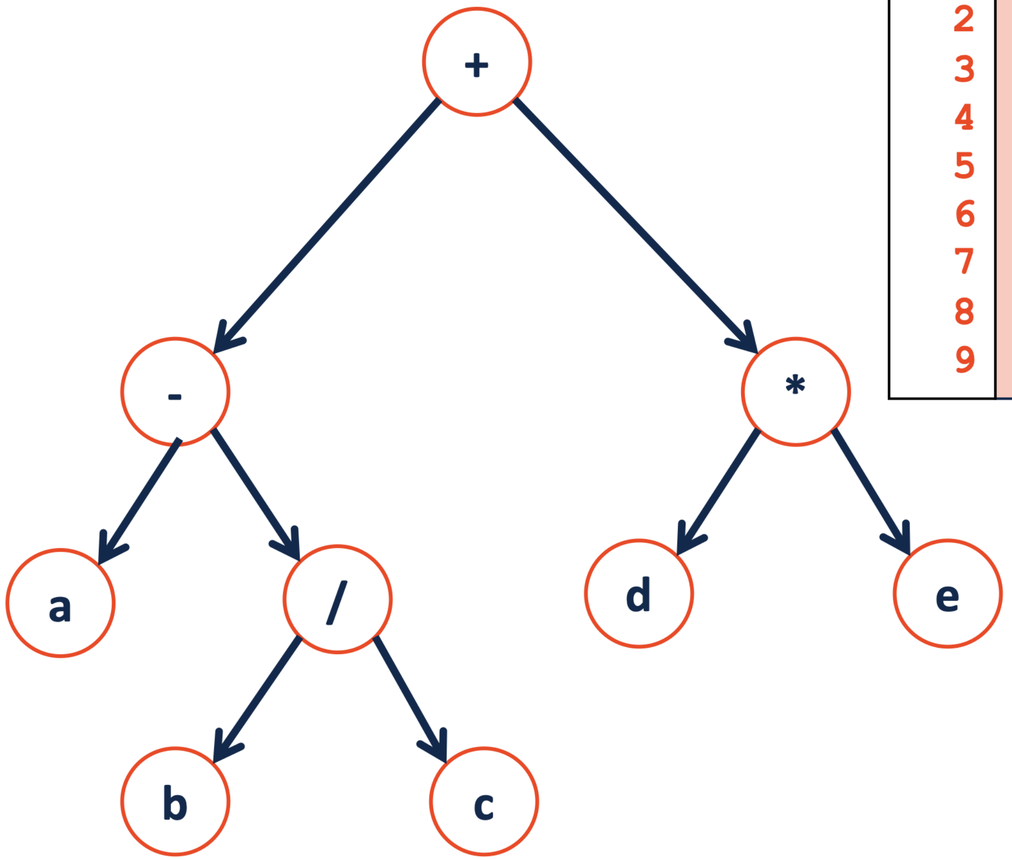



```
1  template<class T>
2  void BinaryTree<T>::preOrder(TreeNode * root)
3  {
4     if (root != NULL) {
5
6        ____print(root)____;
7
8        preOrder(root->left);
9
10       ____________________;
11
12       preOrder(root->right);
13
14       ____________________;
15
16    }
17 }
```

+, −, a, /, b, c, *, d, e

Self, L, R

Traversals




```
1  template<class T>
2  void BinaryTree<T>::preOrder(TreeNode * root)
3  {
4    if (root != NULL) {
5      yell( root->data );
6      preOrder(root->left);
7      preOrder(root->right);
8    }
9  }
```

⇒ ??

Pre : Data [self] , L , R

In : L , Self , R

Post : L , R , Self

```cpp
#include <bits/stdc++.h>
using namespace std;
struct Node {
    int data;
    struct Node *left, *right;
};
// Utility function to create a new tree node
Node* newNode(int data)
{
    Node* temp = new Node;
    temp->data = data;
    temp->left = temp->right = NULL;
    return temp;
}
```

```cpp
/* Given a binary tree, print its nodes in inorder*/
void printInorder(struct Node* node)
{
    if (node == NULL)
        return;
    /* first recur on left child */
    printInorder(node->left);
    /* then print the data of node */
    cout << node->data << " ";
    /* now recur on right child */
    printInorder(node->right);
}
```

```cpp
int main()
{
    struct Node* root = newNode(1);
    root->left = newNode(2);
    root->right = newNode(3);
    root->left->left = newNode(4);
    root->left->right = newNode(5);
    // Function call
    cout << "\nInorder traversal of binary tree is \n";
    printInorder(root);
    return 0;
```

```
Inorder traversal of binary tree is
4 2 5 1 3
```

```cpp
#include <bits/stdc++.h>
using namespace std;
struct Node {
    int data;
    struct Node *left, *right;
};
// Utility function to create a new tree node
Node* newNode(int data)
{
    Node* temp = new Node;
    temp->data = data;
    temp->left = temp->right = NULL;
    return temp;
}
```

```cpp
/* Given a binary tree, print its nodes in preorder*/
void printPreorder(struct Node* node)
{
    if (node == NULL)
        return;

    /* first print data of node */
    cout << node->data << " ";

    /* then recur on left subtree */
    printPreorder(node->left);

    /* now recur on right subtree */
    printPreorder(node->right);
}
```

```cpp
int main()
{
    struct Node* root = newNode(1);
    root->left = newNode(2);
    root->right = newNode(3);
    root->left->left = newNode(4);
    root->left->right = newNode(5);
    // Function call
    cout << "\nInorder traversal of binary tree is \n";
    printInorder(root);
    return 0;
```

```
Preorder traversal of binary tree is
1 2 4 5 3
```

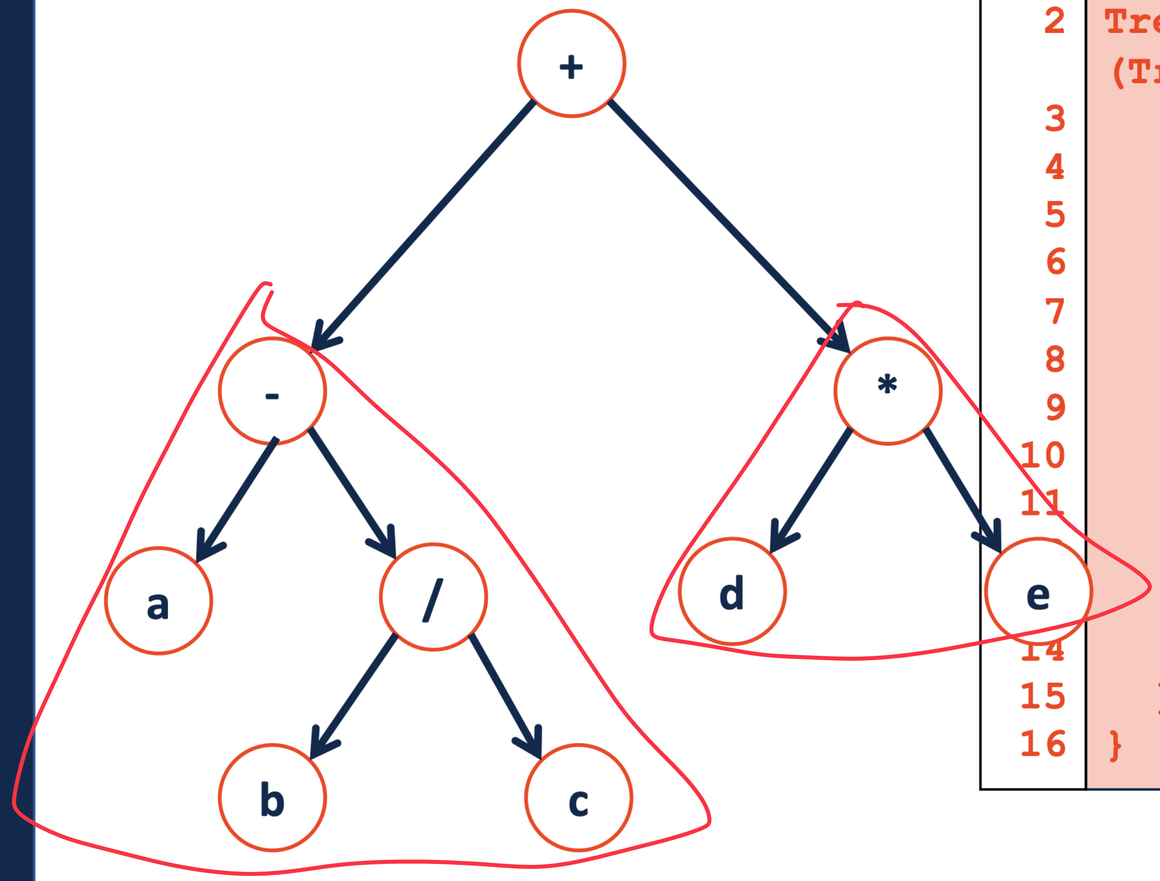
```
#include <bits/stdc++.h>
using namespace std;
struct Node {
    int data;
    struct Node *left, *right;
};
// Utility function to create a new tree node
Node* newNode(int data)
{
    Node* temp = new Node;
    temp->data = data;
    temp->left = temp->right = NULL;
    return temp;
}
```

```
int main()
{
    struct Node* root = newNode(1);
    root->left = newNode(2);
    root->right = newNode(3);
    root->left->left = newNode(4);
    root->left->right = newNode(5);
    // Function call
    cout << "\nInorder traversal of binary tree is \n";
    printInorder(root);
    return 0;
```

```
/* Given a binary tree, print its nodes according to the
"bottom-up" postorder traversal. */
void printPostorder(struct Node* node)
{
    if (node == NULL)
        return;

    // first recur on left subtree
    printPostorder(node->left);

    // then recur on right subtree
    printPostorder(node->right);

    // now deal with the node
    cout << node->data << " ";
}
```

```
Postorder traversal of binary tree is
4 5 2 3 1
```

A Broader View



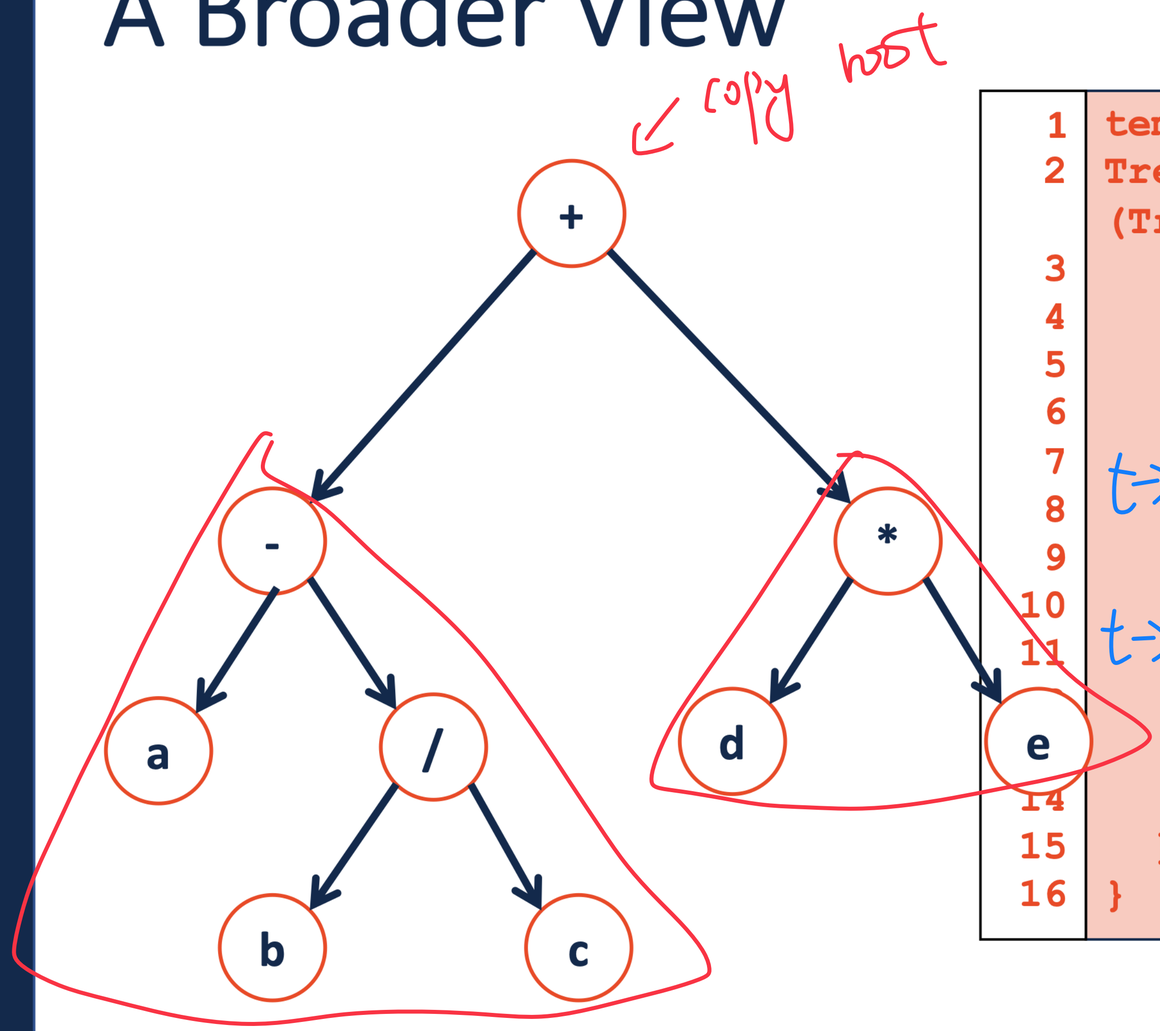

```
1  template<class T>
2  TreeNode * BinaryTree<T>::copy_
   (TreeNode * root) {
3     if (root != NULL) {
4
5
6
7
8
9
10
11
14
15    }
16 }
```

Recursive

_copy(root->left);

_copy(root->right);

A Broader View

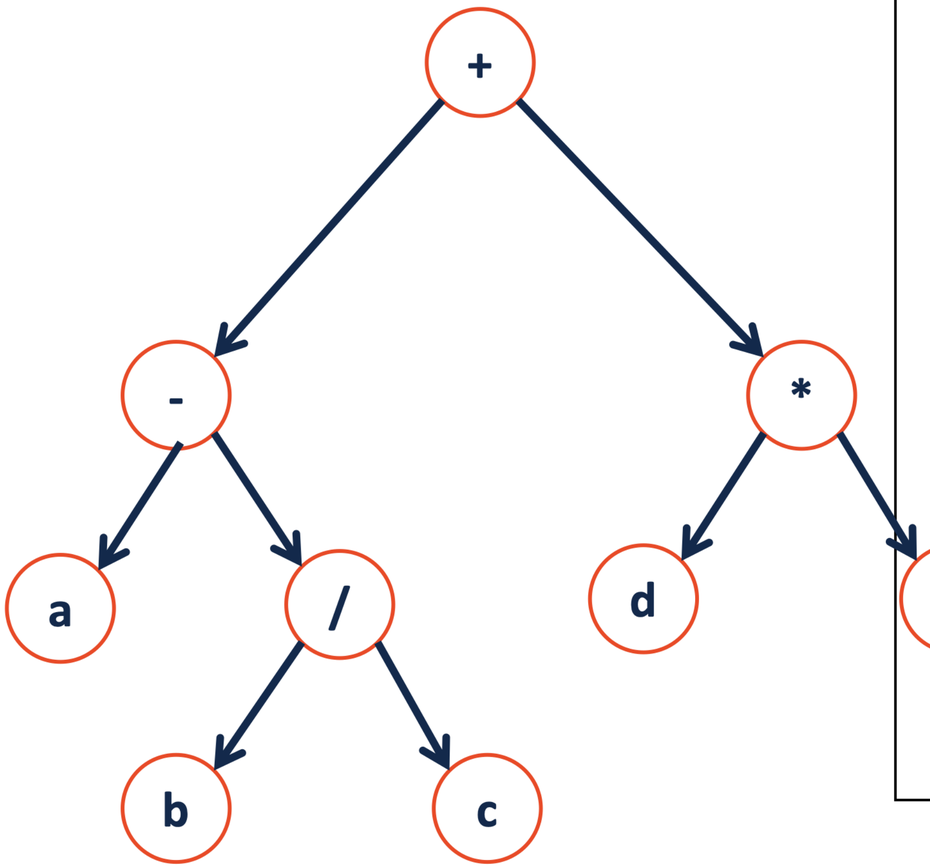



```
1  template<class T>
2  TreeNode * BinaryTree<T>::copy_
   (TreeNode * root) {
3     if (root != NULL) {
4
5        TreeNode * t = new TreeNode(root->data);
6
7     t->left = _copy ( root -> left ) ;
8
9
10    t->right =_copy ( root -> right);
11
14
15    }
16 }   return t ;
```

Recursive

A Broader View

traverse the tree and remove each node
↳ left, right, current

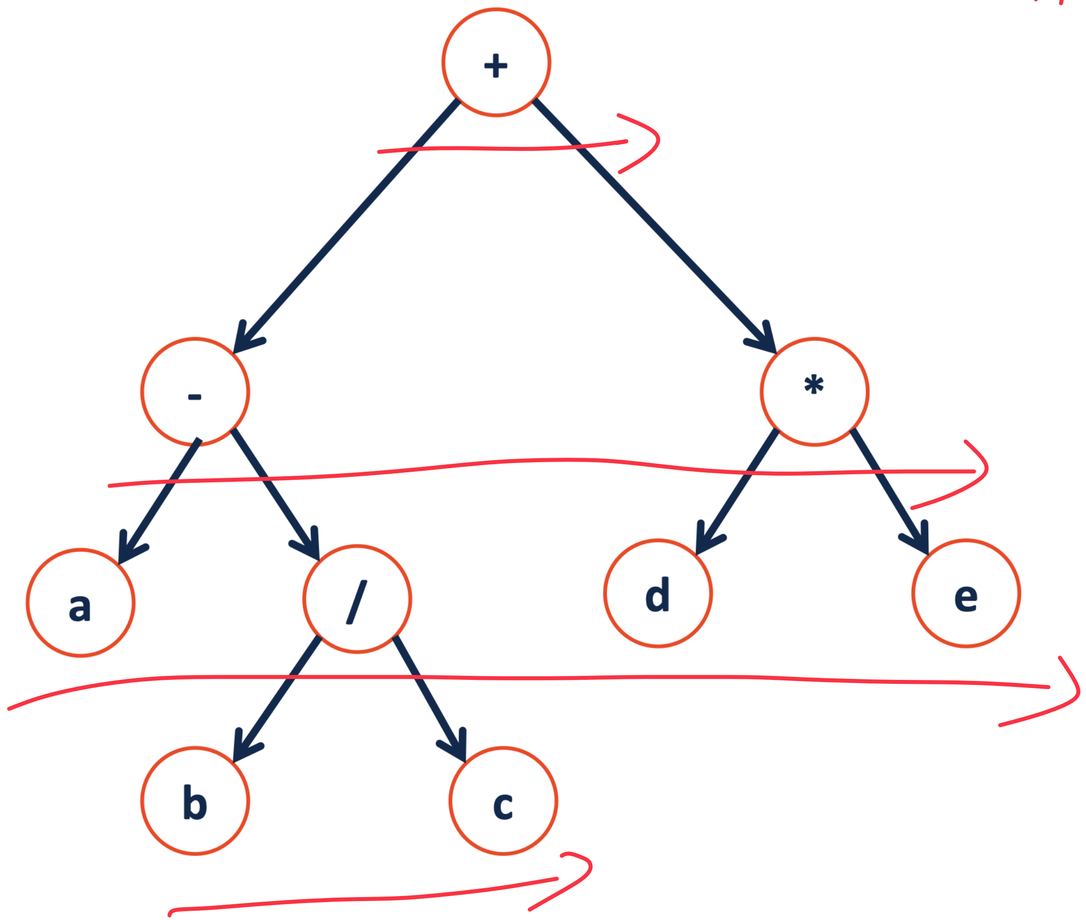



```
1  template<class T>
2  void BinaryTree<T>::clear_(TreeNode * root) {
3     if (root != NULL) {
4
5
6              clear_ (root → left);
7
8              clear_ (root → right);
9
10             delete root;
11
12
13
14
15
16    }
17 }
```

A Different Type of Traversal

→ level - by - level

+, -, *, a, /, d, e

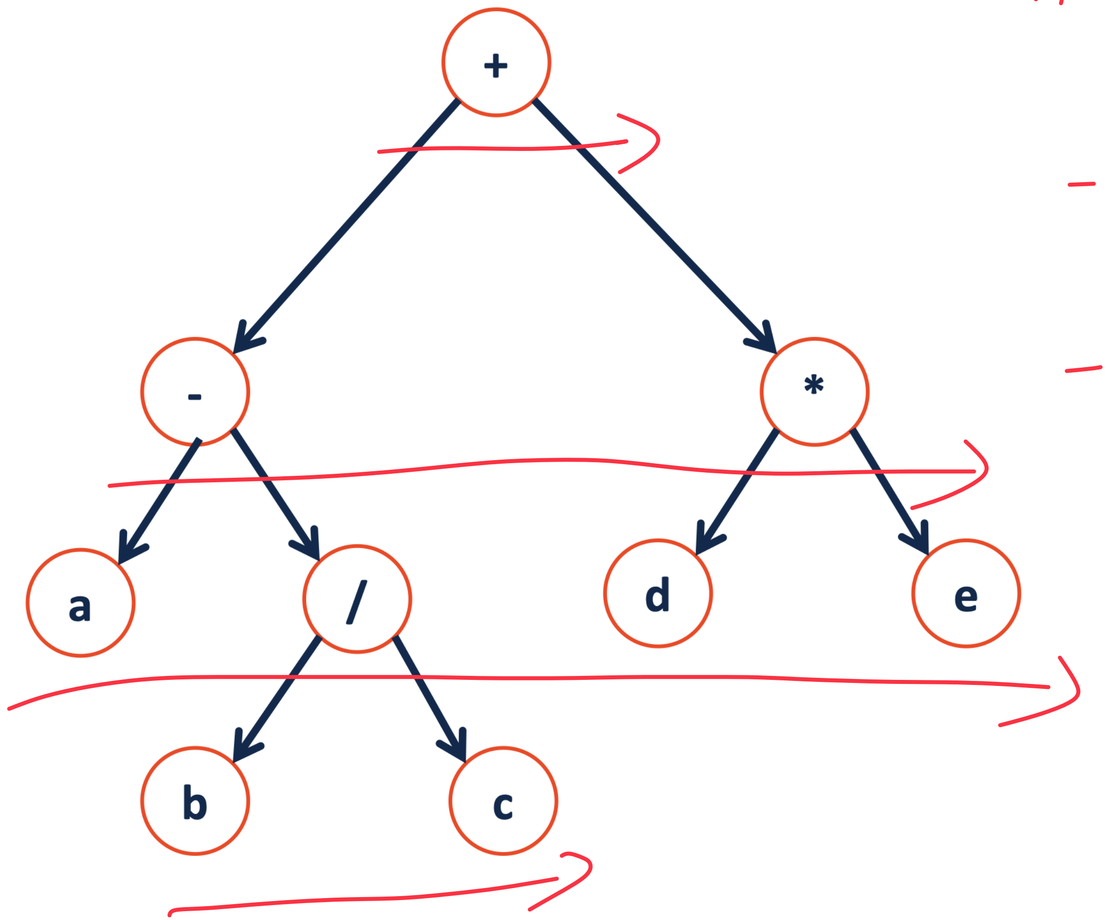


A Different Type of Traversal

→ level-by-level

+, -, *, a, /, d, e

How??



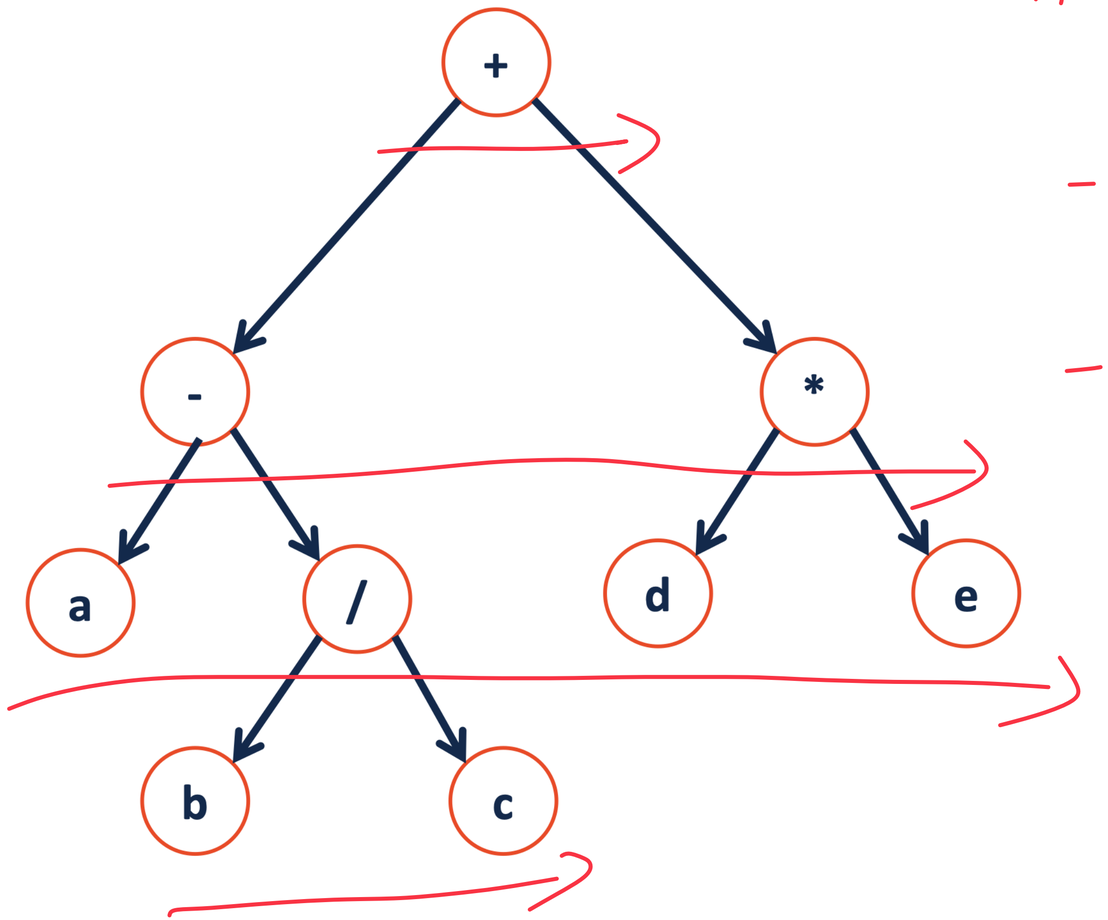
- put the first node to a queue
- while the queue is not empty:
  1. remove element from queue
  2. if not null:
     - print value
     - enqueue its children

A Different Type of Traversal

→ level-by-level

+, -, *, a, /, d, e

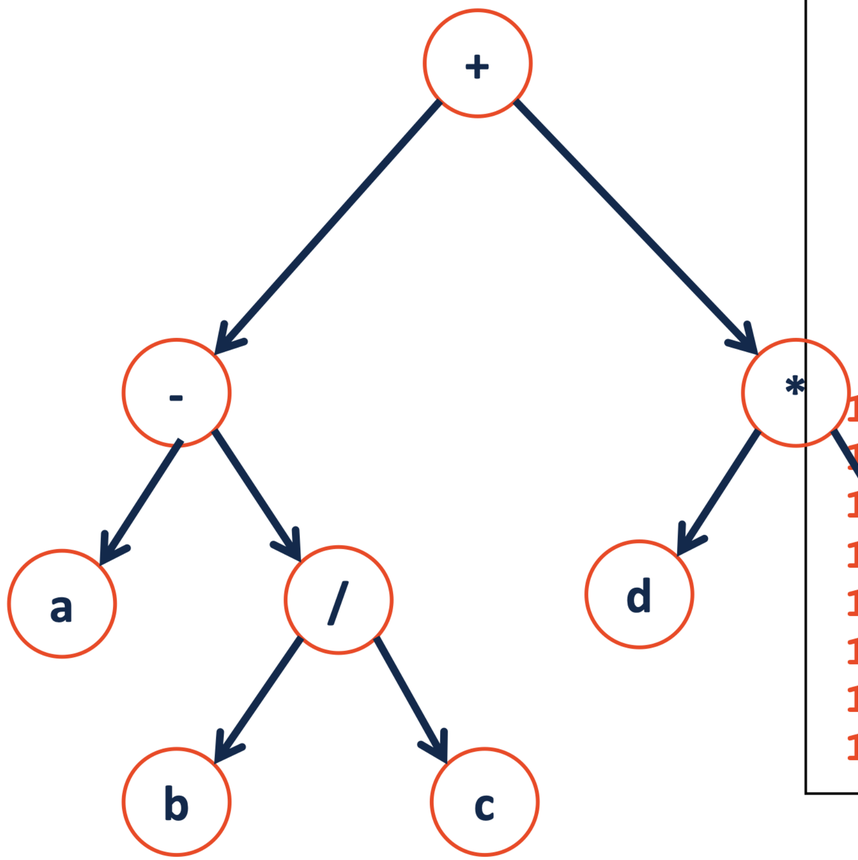
How??



- put the first node to a queue
- while the queue is not empty:
  ① remove element from queue
  ② if not null:
    - print value
    - enqueue its children

Recursion?? → No

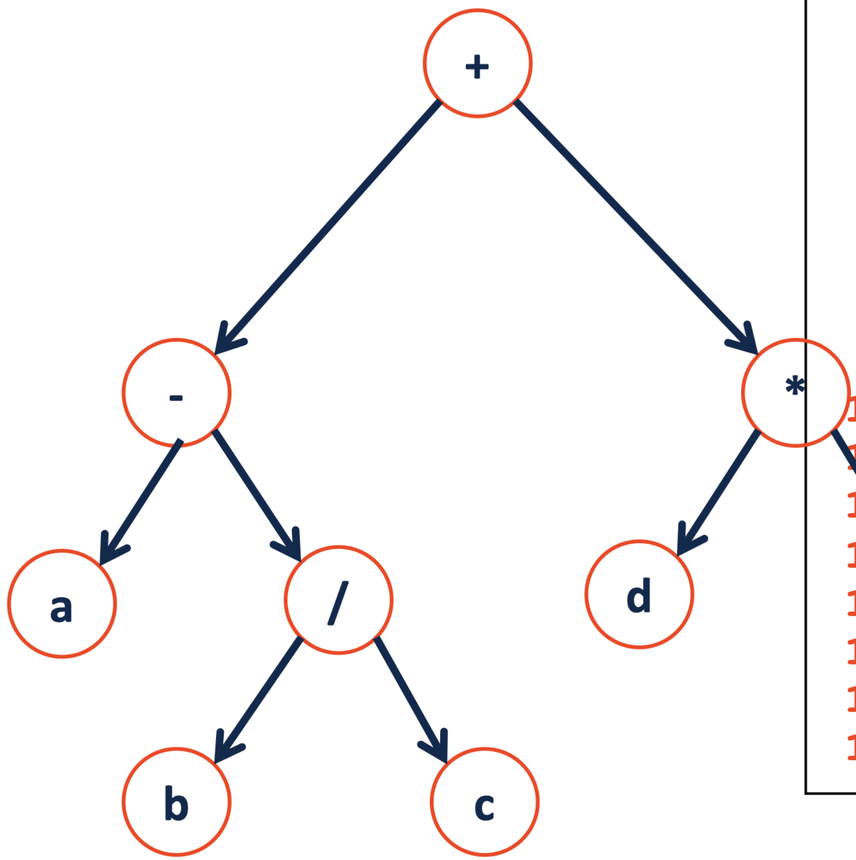
A Different Type of Traversal




```
1  template<class T>
2  void BinaryTree<T>::levelOrder(TreeNode * root) {
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17 }
```

1. + → +
2. - * → - *
3. * a / → * a /
a / d e →
⋮

A Different Type of Traversal




```
1  template<class T>
2  void BinaryTree<T>::levelOrder(TreeNode * root) {
3
4     Queue<TreeNode *> q;
5
6     q.enqueue(root);
7
8     while(!q.isEmpty()) {
9
10          TreeNode *t = q.dequeue();
11
12          if(t != Null) {
13
14             yes(t->data);
15             q.enqueue(t->left);
16             q.enqueue(t->right);
17          }
    }
```

Traversal vs. Search

Traversal

Visit Every Node Exactly Once

Search

Visit Every Node until we find

one or more matches!

Search: Breadth First vs. Depth First

Strategy: Breadth First Search (BFS)

a vertex-based technique for finding the shortest path in the graph. It uses a Queue data structure that follows first in first out. In BFS, one vertex is selected at a time when it is visited and marked then its adjacent are visited and stored in the queue. It is slower than DFS.

Strategy: Depth First Search (DFS)

an edge-based technique. It uses the Stack data structure and performs two stages, first visited vertices are pushed into the stack, and second if there are no vertices then visited vertices are popped.

Difference Between BFS and DFS:

Parameters	BFS	DFS
Stands for	BFS stands for Breadth First Search.	DFS stands for Depth First Search.
Data Structure	BFS(Breadth First Search) uses Queue data structure for finding the shortest path.	DFS(Depth First Search) uses Stack data structure.
Definition	BFS is a traversal approach in which we first walk through all nodes on the same level before moving on to the next level.	DFS is also a traversal approach in which the traverse begins at the root node and proceeds through the nodes as far as possible until we reach the node with no unvisited nearby nodes.
Conceptual Difference	BFS builds the tree level by level.	DFS builds the tree sub-tree by sub-tree.
Approach used	It works on the concept of **FIFO (First In First Out)**.	It works on the concept of **LIFO (Last In First Out).**
Suitable for	BFS is more suitable for searching vertices closer to the given source.	DFS is more suitable when there are solutions away from source.

Dictionary ADT

Data is often organized into key/value pairs:

UIN → Advising Record
Course Number → Lecture/Lab Schedule
Node → Incident Edges
Flight Number → Arrival Information
URL → HTML Page
…

Dictionary.h

```
1  #ifndef DICTIONARY_H
2  #define DICTIONARY_H
3
4  template < class K , class V >
5  class Dictionary {
6    public:
7
8            void Insert( K key, V value );
9
10           void remove(const K& key);
11
12           V find(const K& key) const;
13
14           Dictionary( );
15
16
17
18
19   private:
20 };
21
22 #endif
```

key may be very complicated.

CS 225

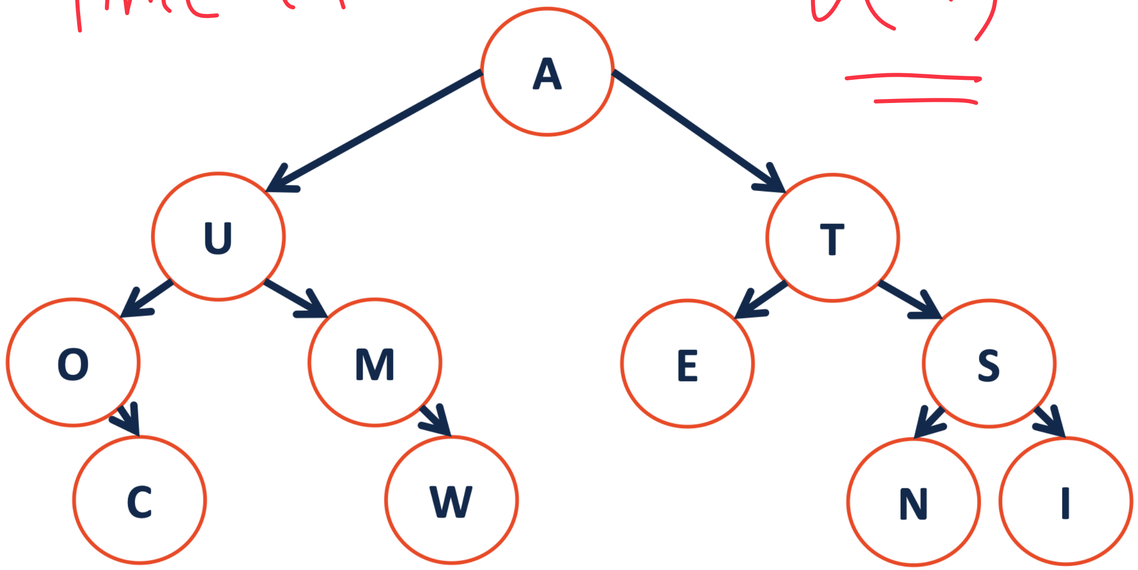
Data Structures

Volodymyr Kindratenko

Slides courtesy of Wade Fagen-Ulmschneider

Binary Tree as a Search Structure

find('w') Time ?? O(n)



do better ??

Binary Search Tree (BST)

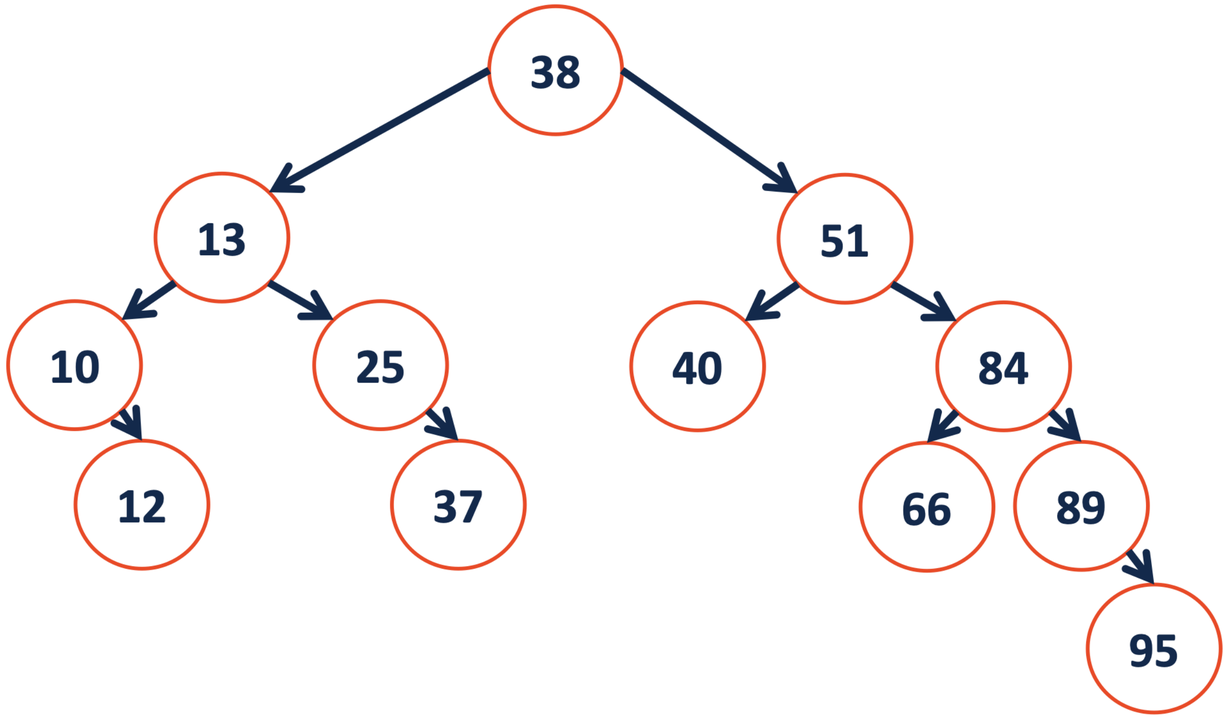
A **BST** is a binary tree **T** such that:

$T = \{\}$ or

$T = \{r, T_L, T_R\}$ and

- x in T_L if $x < r$
- x in T_R if $x > r$

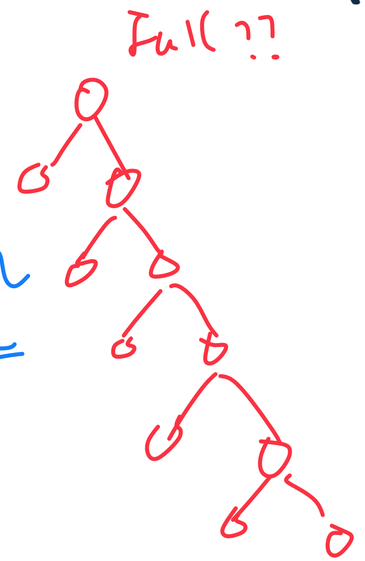
find(x)



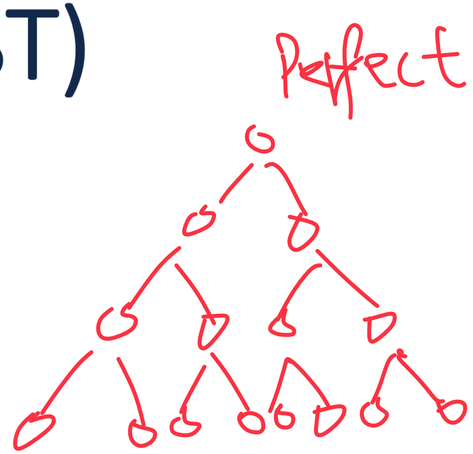
Binary Search Tree (BST)

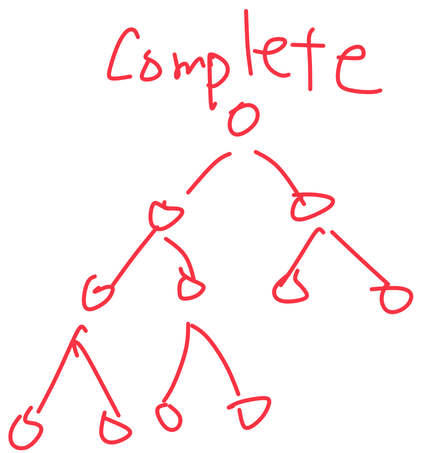
Best case?

- root
- height ~ lg n



Worst case?

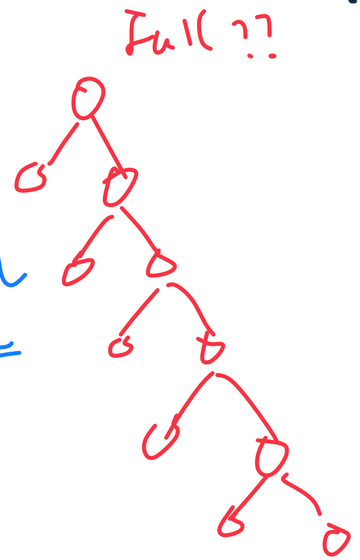




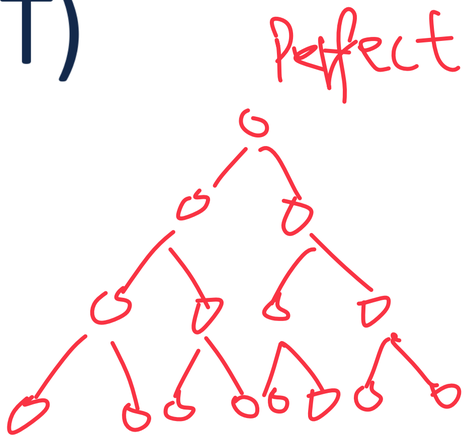
Binary Search Tree (BST)

Best case?

- root
- height ~ lg n



Worst case?

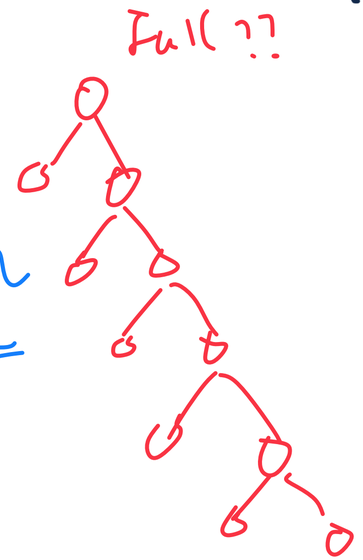


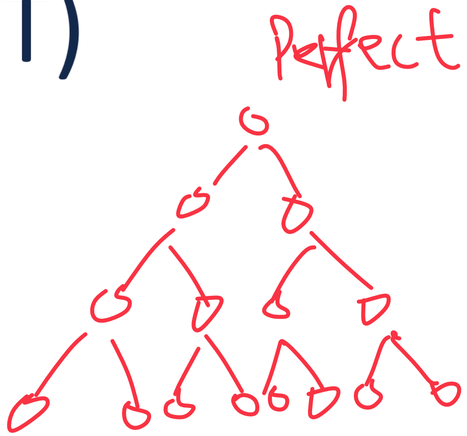
$2^{h+1} - 1 \Rightarrow h \sim \log n$

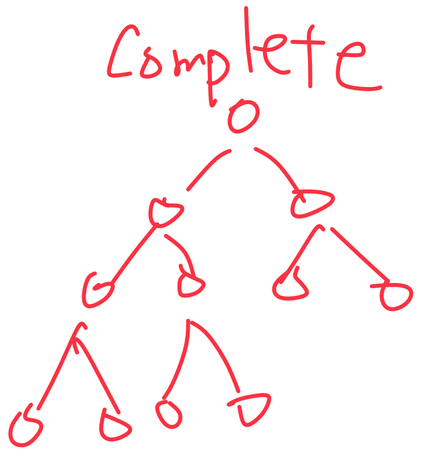
Binary Search Tree (BST)

Best case?

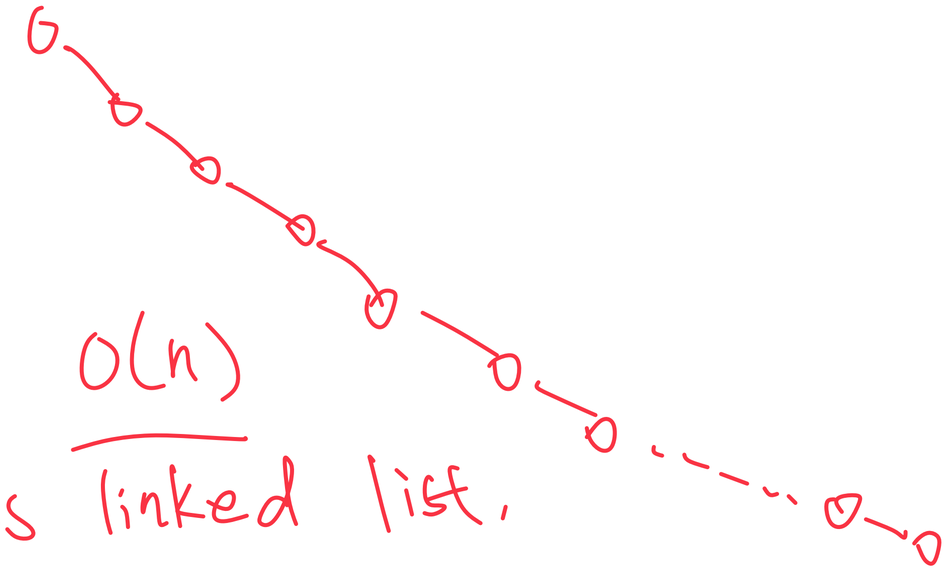
- root
- height ~ lg n







Worst case?

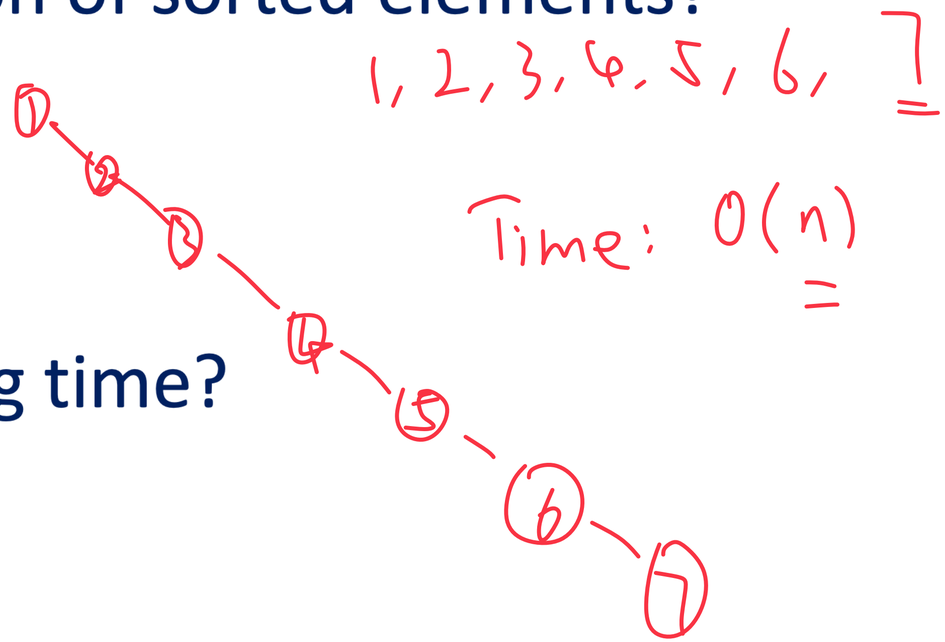


O(n)

as linked list.

Binary Search Tree (BST)

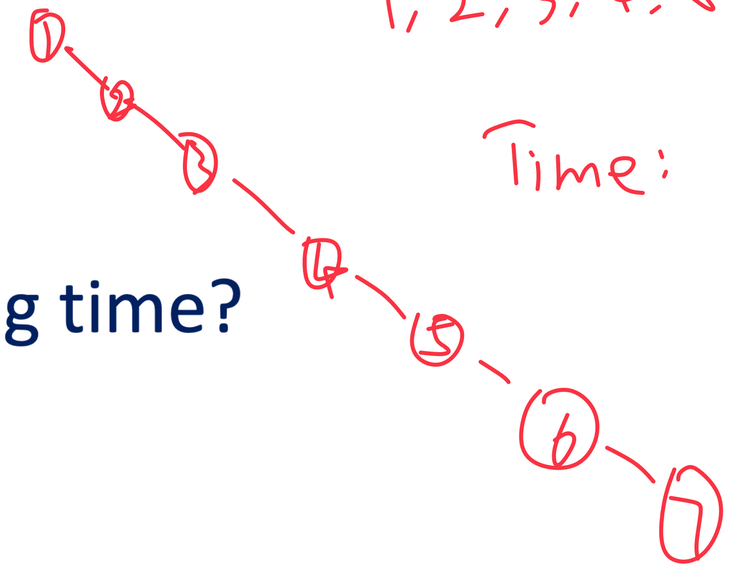
Insertion of sorted elements?



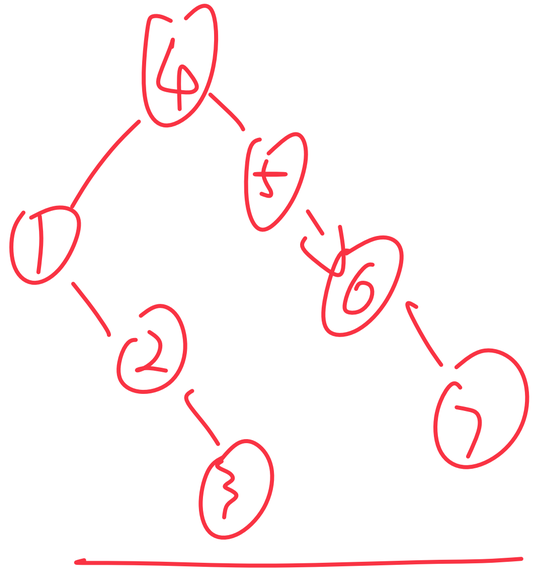
Running time?

Binary Search Tree (BST)

Insertion of sorted elements?



Running time?



better??

O(n/2)

BST.h

```
 1  #ifndef DICTIONARY_H
 2  #define DICTIONARY_H
 3
 4  template <class K, class V>
 5  class BST {
 6    public:
 7      BST();
 8      void insert(const K key, V value);
 9      V remove(const K & key);
10      V find(const K & key) const;
11      TreeIterator traverse() const;
12    private:
13
14
15
16
17
18
19
20  };
21
22  #endif
```

struct TreeNode{

TreeNode *left, *right;

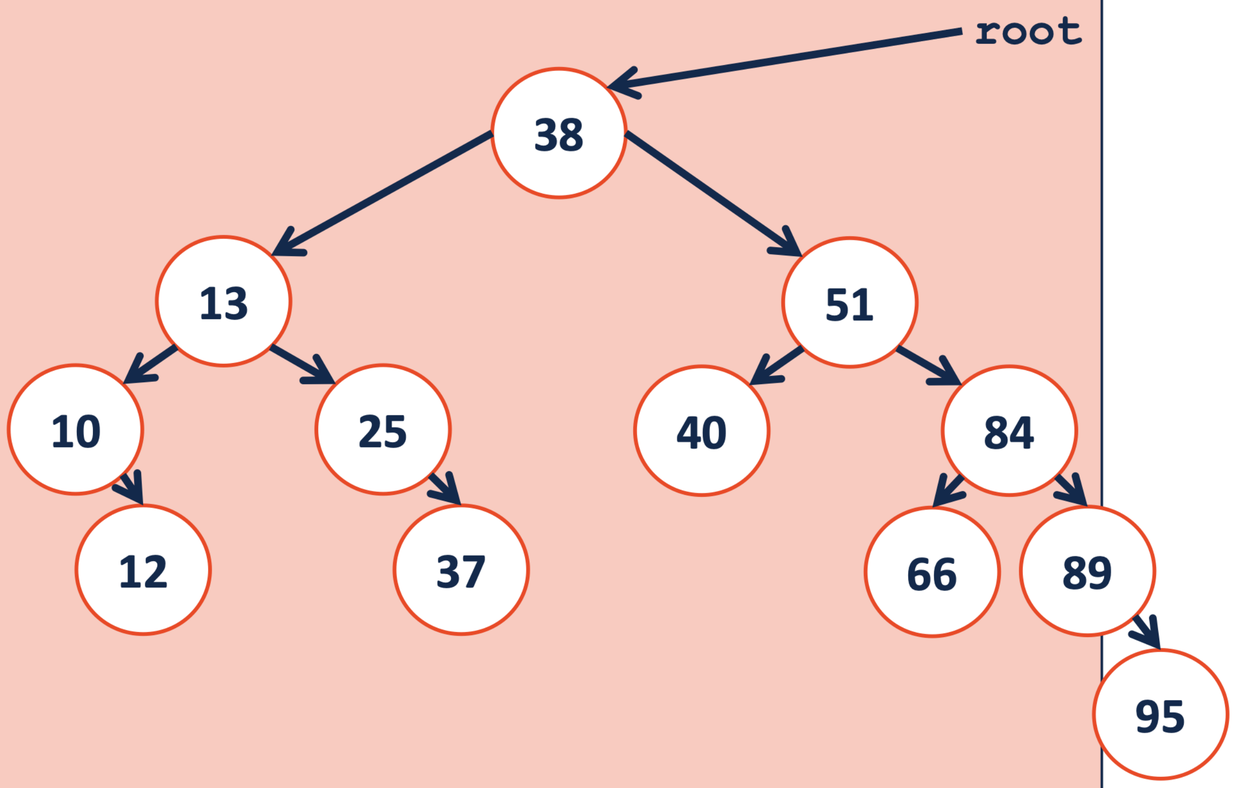
V value; K key; }

TreeNode(K key, V value):

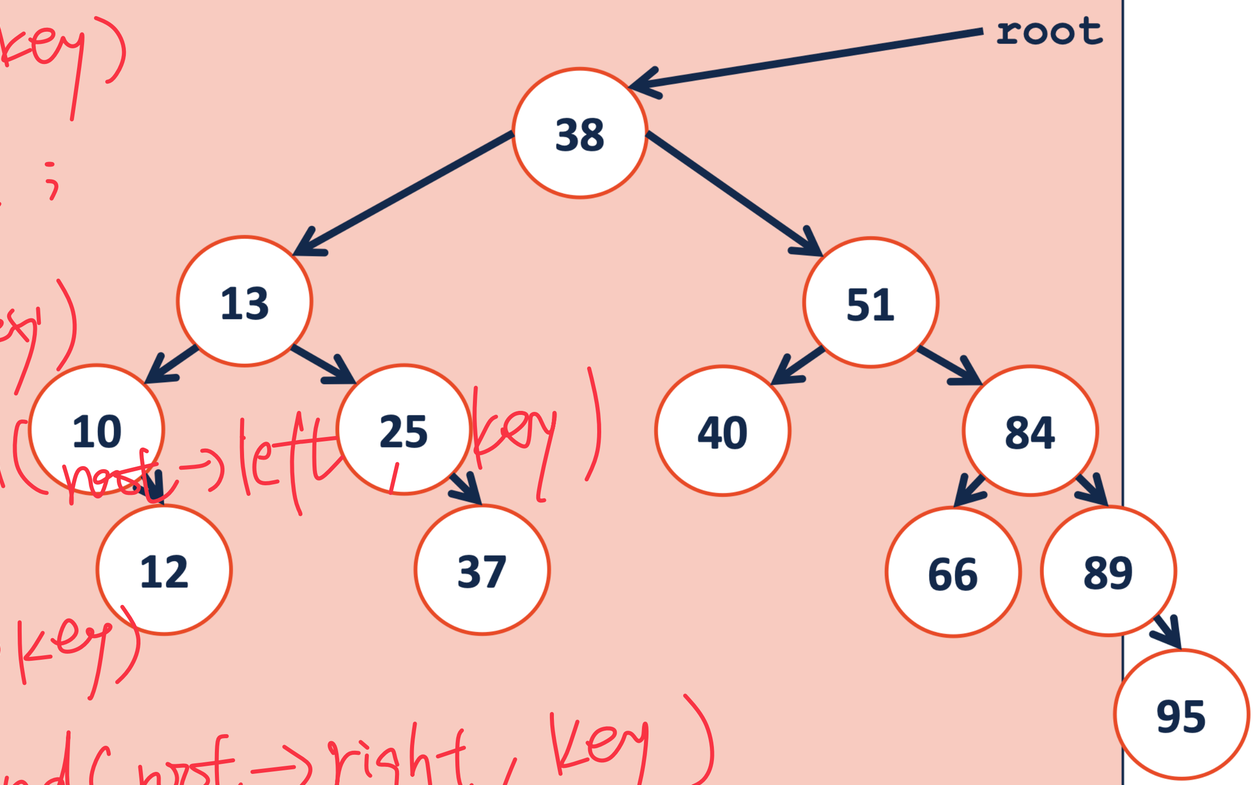
key(key), value(value),

left(null), right(null){ }

```
1  template<class K, class V>
2  TreeNode *& BST::find(TreeNode *& root, const K & key) const {
3
4
5      // Equal to
6
7
8
9      // less than
10
11
12     // greater than
13
14
15
16
17
18
19
20
21
22
23
24
25
26 }
```

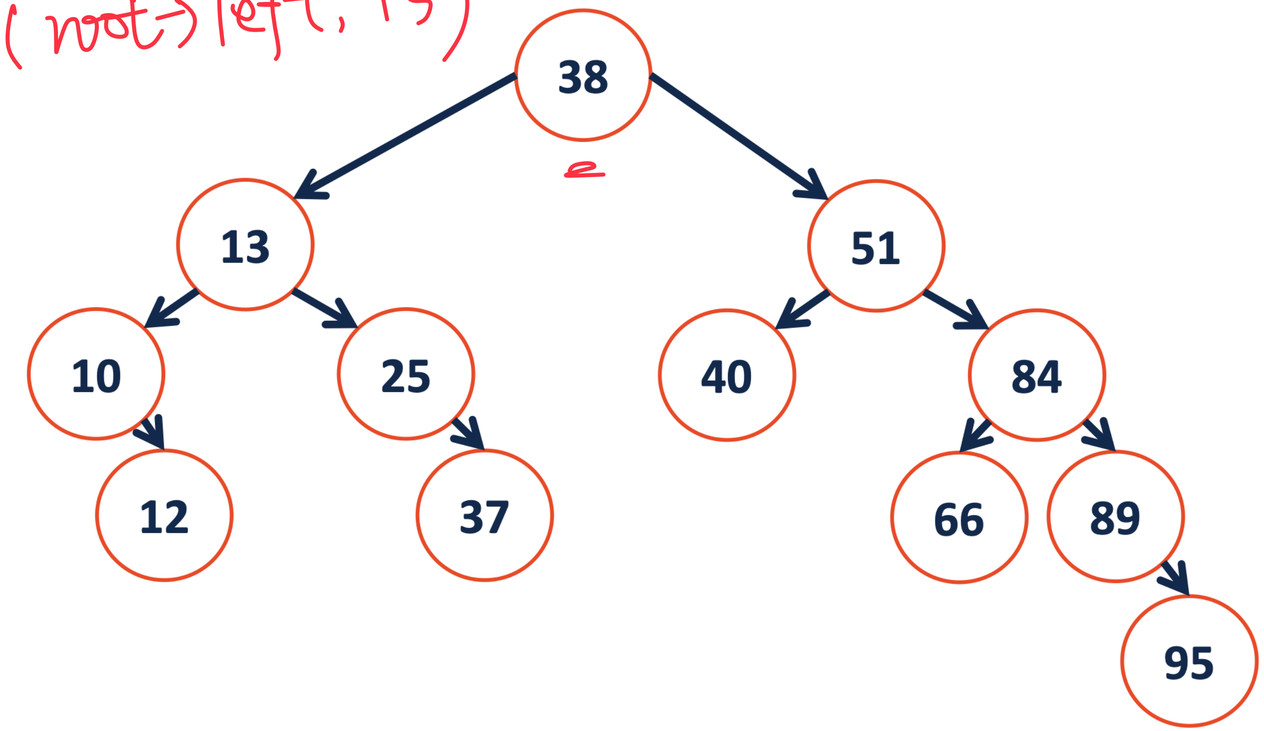


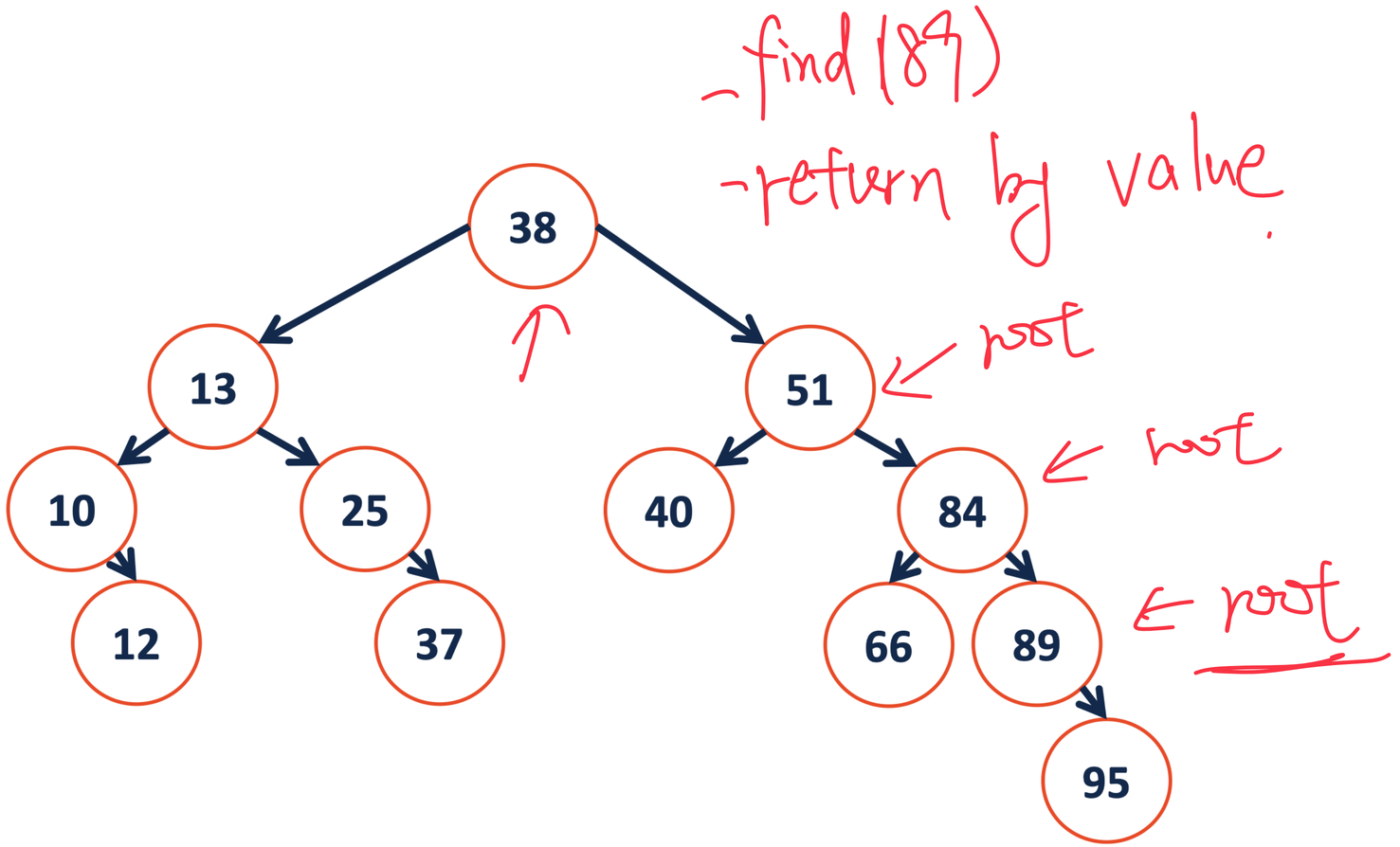
```
1  template<class K, class V>
2  TreeNode *& BST::find(TreeNode *& root, const K & key) const {
3
4      // Base case  root == Null
5      if (root == Null)
6          return root;
7
8      // Equal to
9      if (key == root->key)
10         return root;
11
12     // Less than
13     if (key < root->key)
14         return _find(root->left, key);
15
16     // Equal than
17     if (key > root->key)
18         return _find(root->right, key);
19
20 }
21
22
23
24
25
26
```



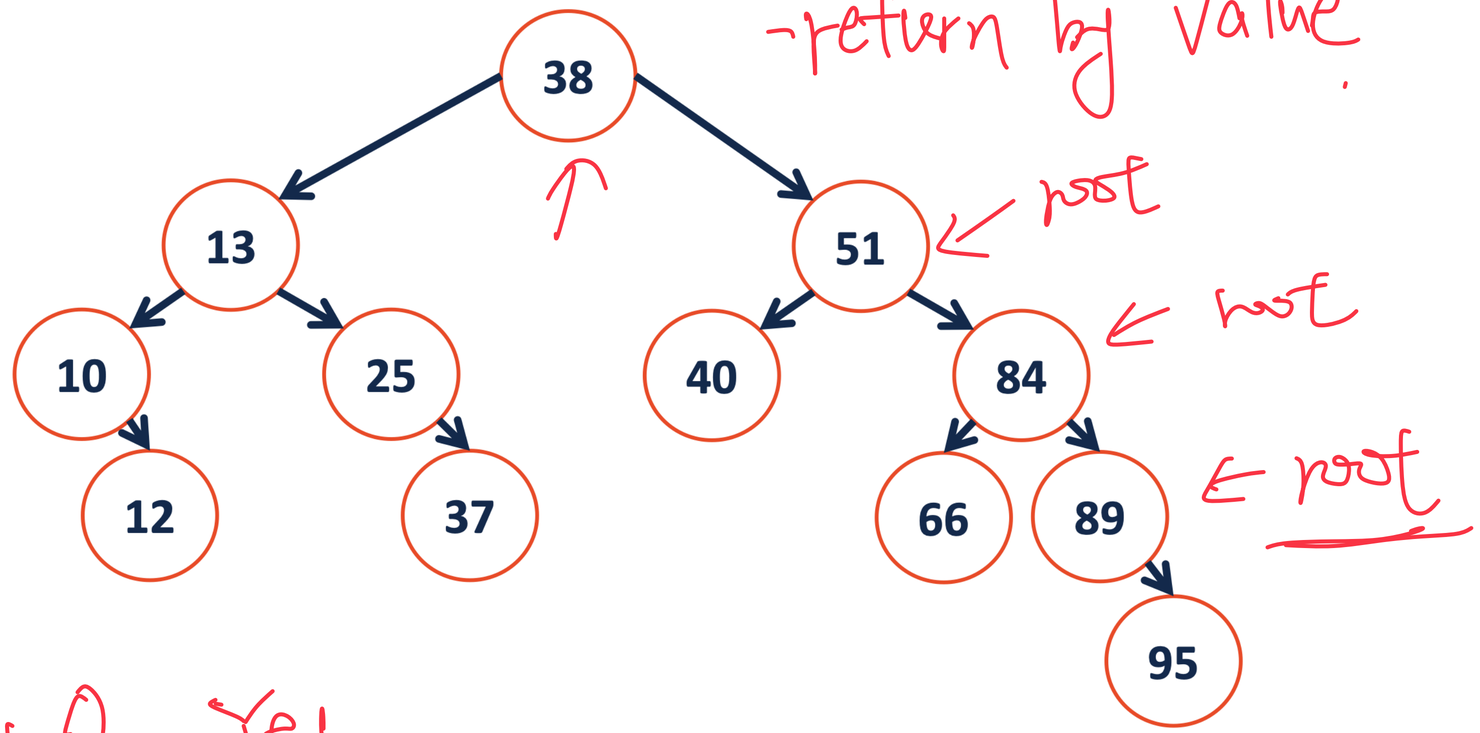
find(root, 13)

find(root->left, 13)

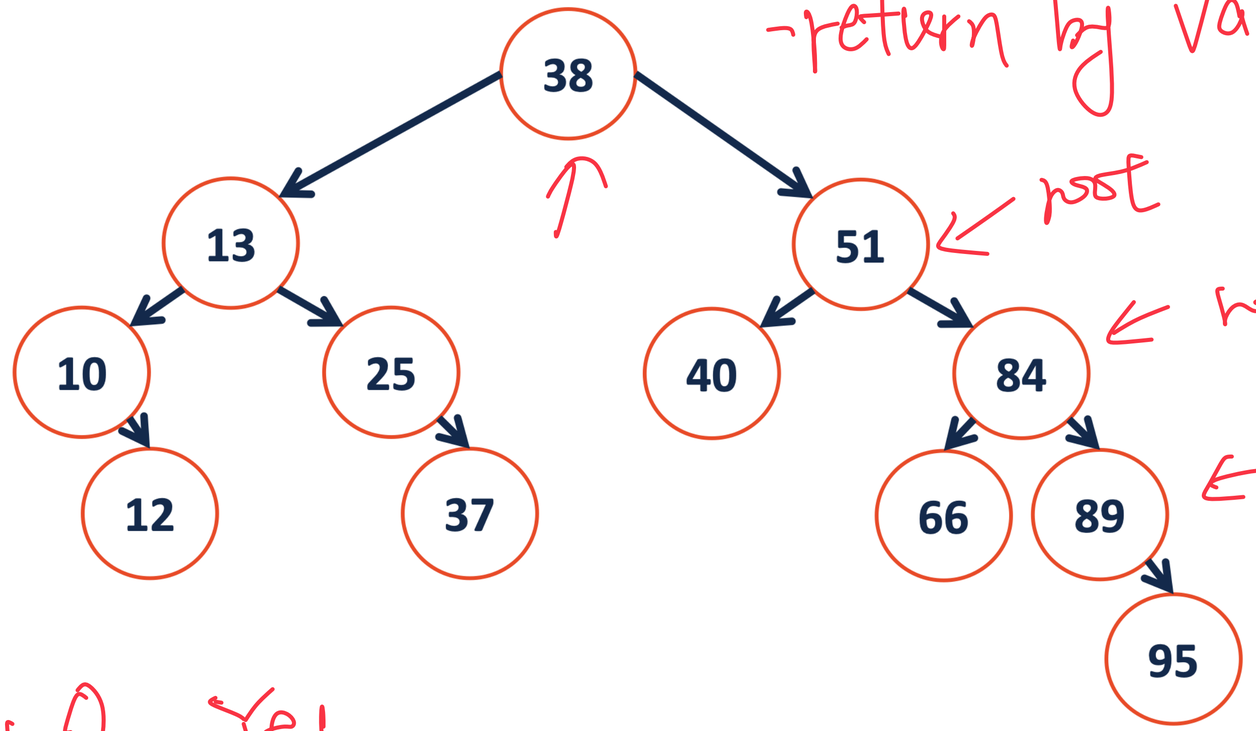



-find(84)
-return by value.
38
13
51
root
10
25
40
84
root
12
37
66
89
root
95

- find(84)
- return by value



- find it? Yes
- Can modify tree?? No



- find(89)
- return by value

root
root
root

- find it? Yes
- Can modify tree?? No

return by reference
→ TreeNode *&

Passing by Pointer - change values pointed to

```cpp
// C++ program to swap two numbers using
// pass by pointer
#include <iostream>
using namespace std;

void swap(int *x, int *y)
{
    int z = *x;
    *x = *y;
    *y = z;
}

// Driver Code
int main()
{
    int a = 45, b = 35;
    cout << "Before Swap\n";
    cout << "a = " << a << " b = " << b << "\n";

    swap(&a, &b);

    cout << "After Swap with pass by pointer\n";
    cout << "a = " << a << " b = " << b << "\n";
}
```

```
Before Swap
a = 45 b = 35
After Swap with pass by pointer
a = 35 b = 45
```

Passing by Pointer - change the pointer itself

```cpp
#include <iostream>

using namespace std;

int global_Var = 42;

// function to change pointer value
void changePointerValue(int* pp)
{
    pp = &global_Var;
}

int main()
{
    int var = 23;
    int* ptr_to_var = &var;

    cout << "Passing Pointer to function:" << endl;

    cout << "Before :" << *ptr_to_var << endl; // display 23

    changePointerValue(ptr_to_var);

    cout << "After :" << *ptr_to_var << endl; // display 23

    return 0;
}
```

“pass by pointer” is passing a pointer by value.

```
Passing Pointer to function:
Before :23
After :23
```

Passing by Reference to a Pointer

```cpp
#include <iostream>

using namespace std;

int gobal_var = 42;

// function to change Reference to pointer value
void changeReferenceValue(int*& pp)
{
    pp = &gobal_var;
}

int main()
{
    int var = 23;
    int* ptr_to_var = &var;

    cout << "Passing a Reference to a pointer to function" << endl;

    cout << "Before :" << *ptr_to_var << endl; // display 23

    changeReferenceValue(ptr_to_var);

    cout << "After :" << *ptr_to_var << endl; // display 42

    return 0;
}
```

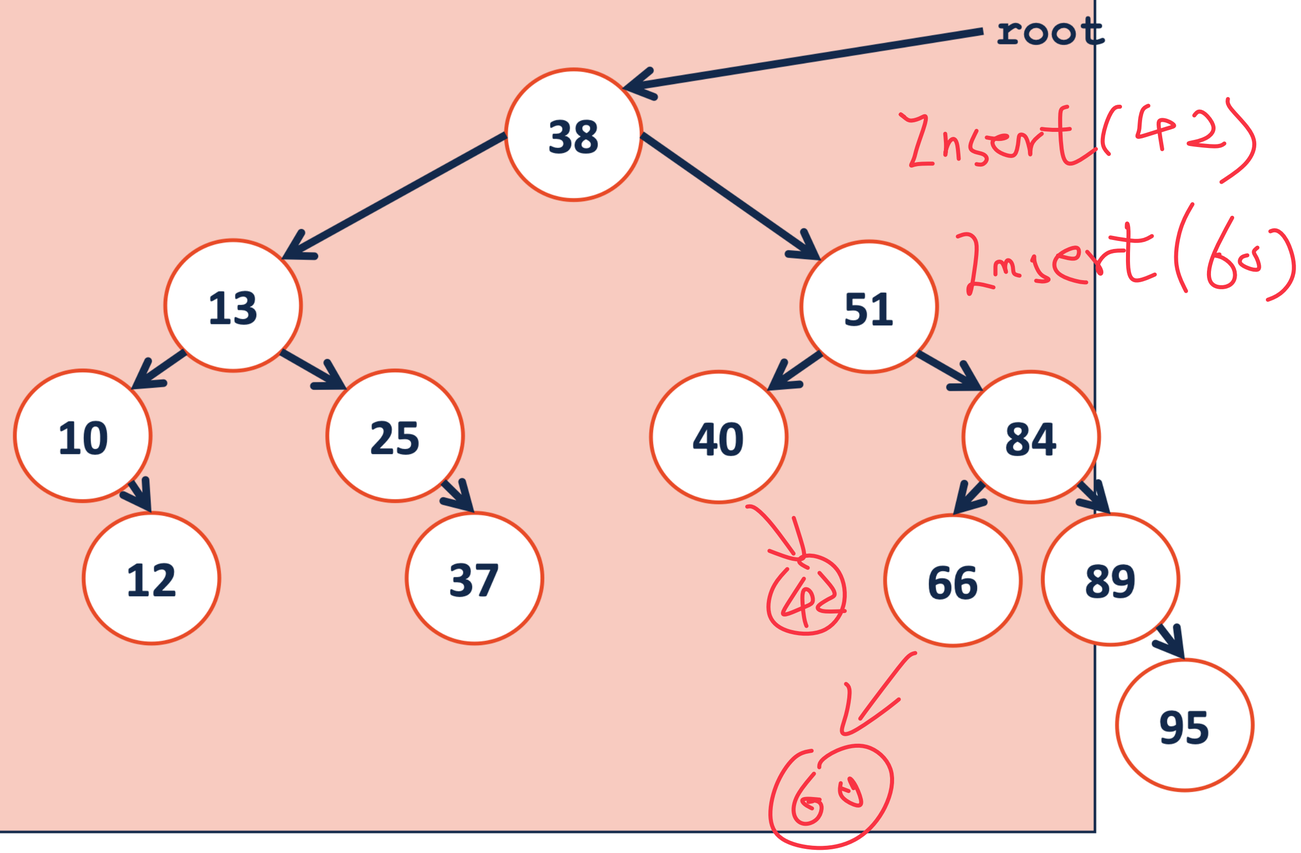
```
Passing a Reference to a pointer to function
Before :23
After :42
```

Passing by Pointer to a Pointer

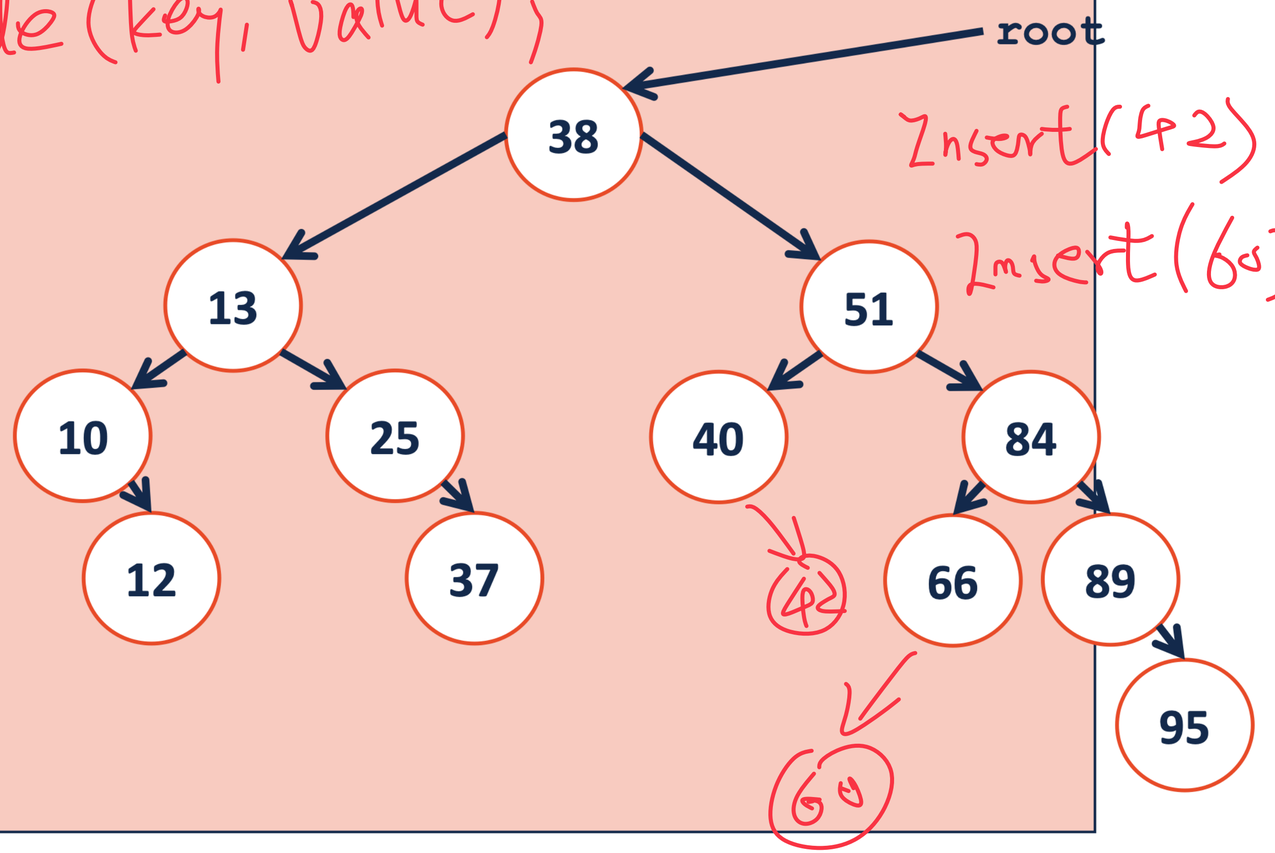
```cpp
#include <iostream>

using namespace std;

int global_var = 42;

// function to change pointer to pointer value
void changePointerValue(int** ptr_ptr)
{
    *ptr_ptr = &global_var;
}

int main()
{
    int var = 23;
    int* pointer_to_var = &var;

    cout << "Passing a pointer to a pointer to function " << endl;

    cout << "Before :" << *pointer_to_var << endl; // display 23

    changePointerValue(&pointer_to_var);

    cout << "After :" << *pointer_to_var << endl; // display 42

    return 0;
}
```

```
Passing a pointer to a pointer to function
Before :23
After :42
```

```
1  template<class K, class V>
2  ____________________________ insert(TreeNode *& root, K key, V value) {
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26 }
```



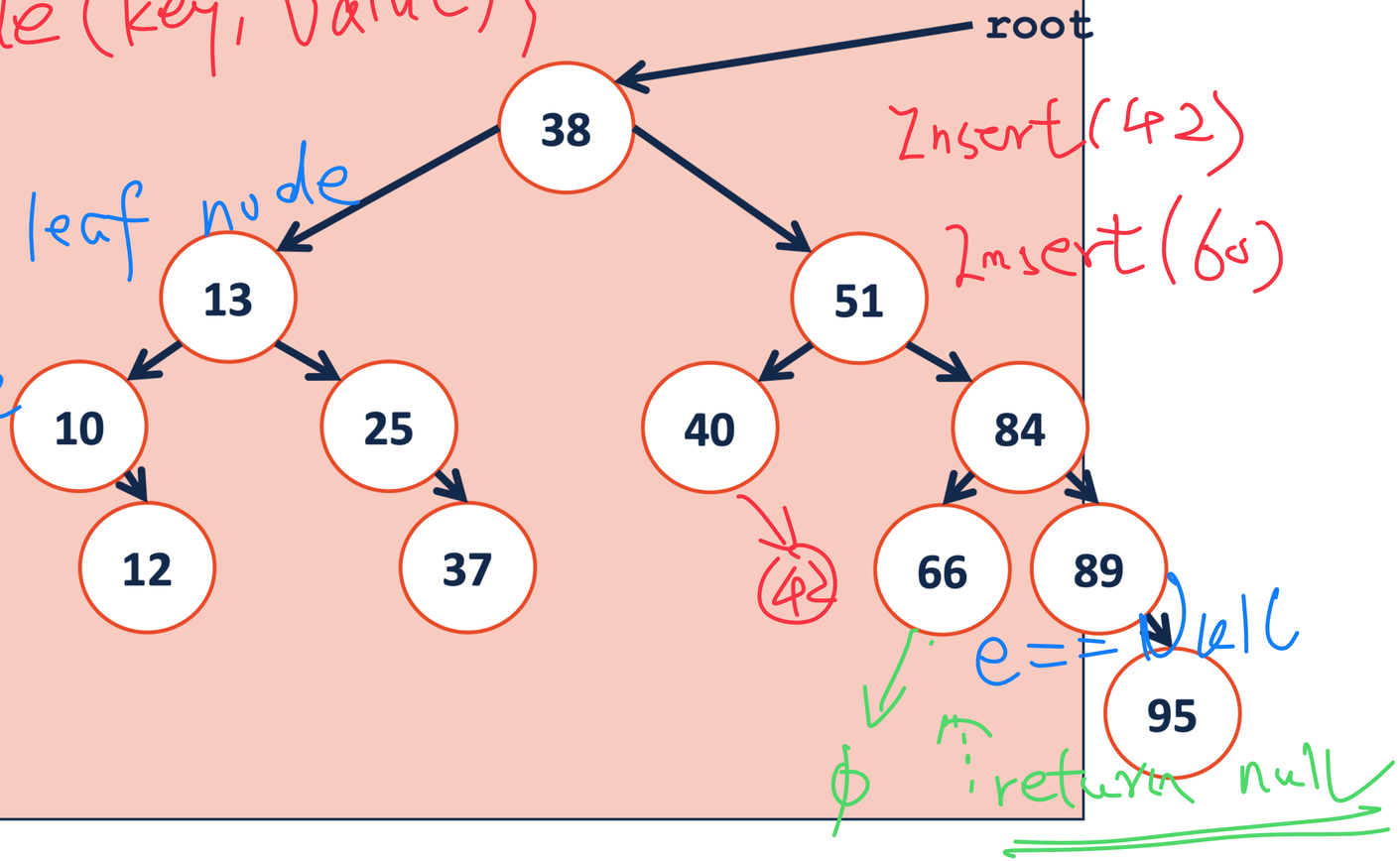
```
1  template<class K, class V>
2  void :: BST                insert(TreeNode *& root, K key, V value) {
3
4      TreeNode *&e = find_(root, key);
5
6      // check if e is null
7
8      e = new TreeNode(key, value);
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26 }
```



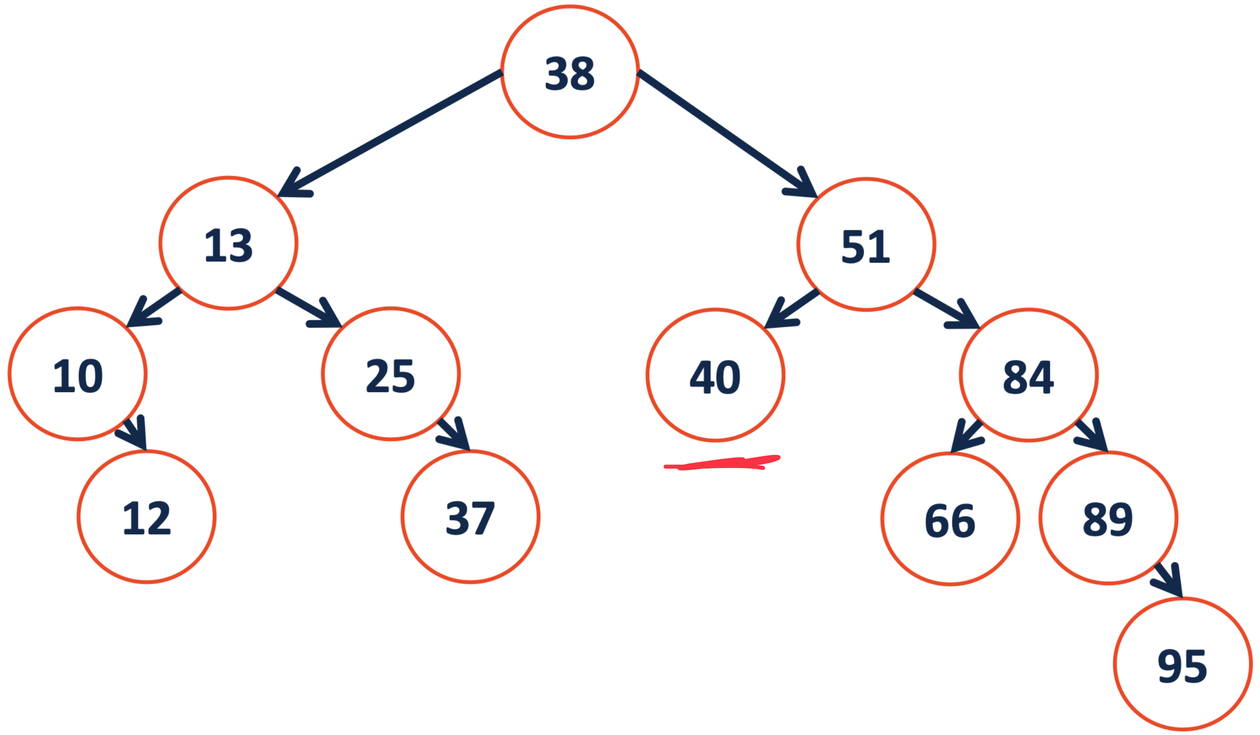
```
1  template<class K, class V>
2  void :: BST ____ insert(TreeNode *& root, K key, V value) {
3
4      TreeNode *&e = find_(root, key);
5
6      // check if e is null
7
8      e = new TreeNode(key, value);
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26 }
```

- Insert will always at leaf node

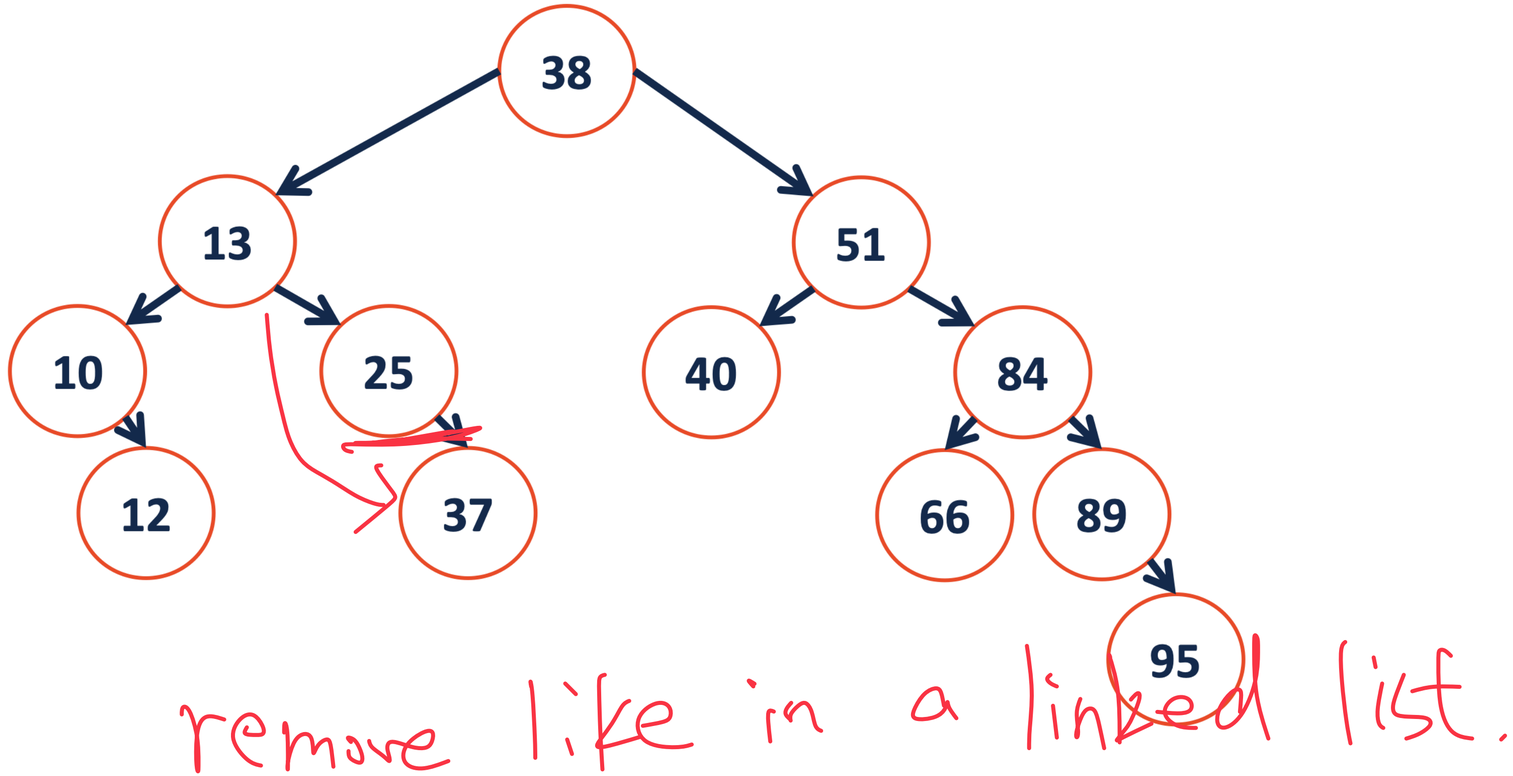
- Inset might increase tree height



Remove:

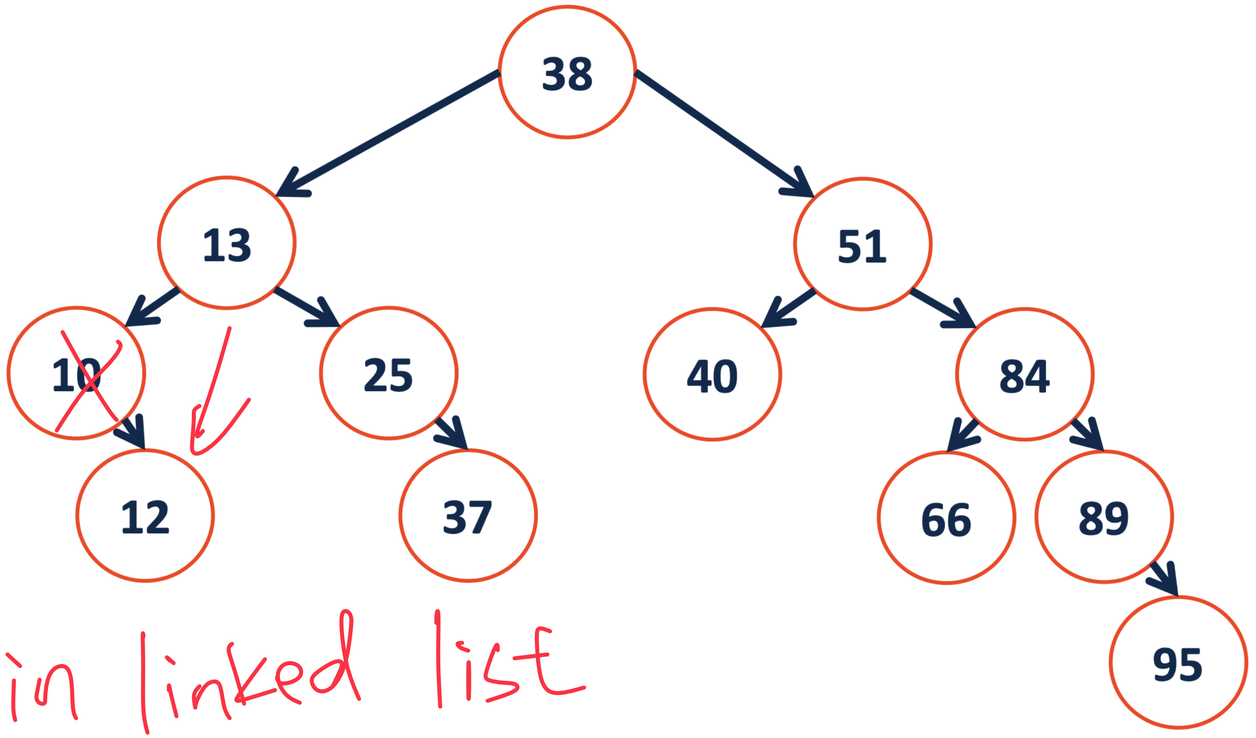



```
remove(40);
```



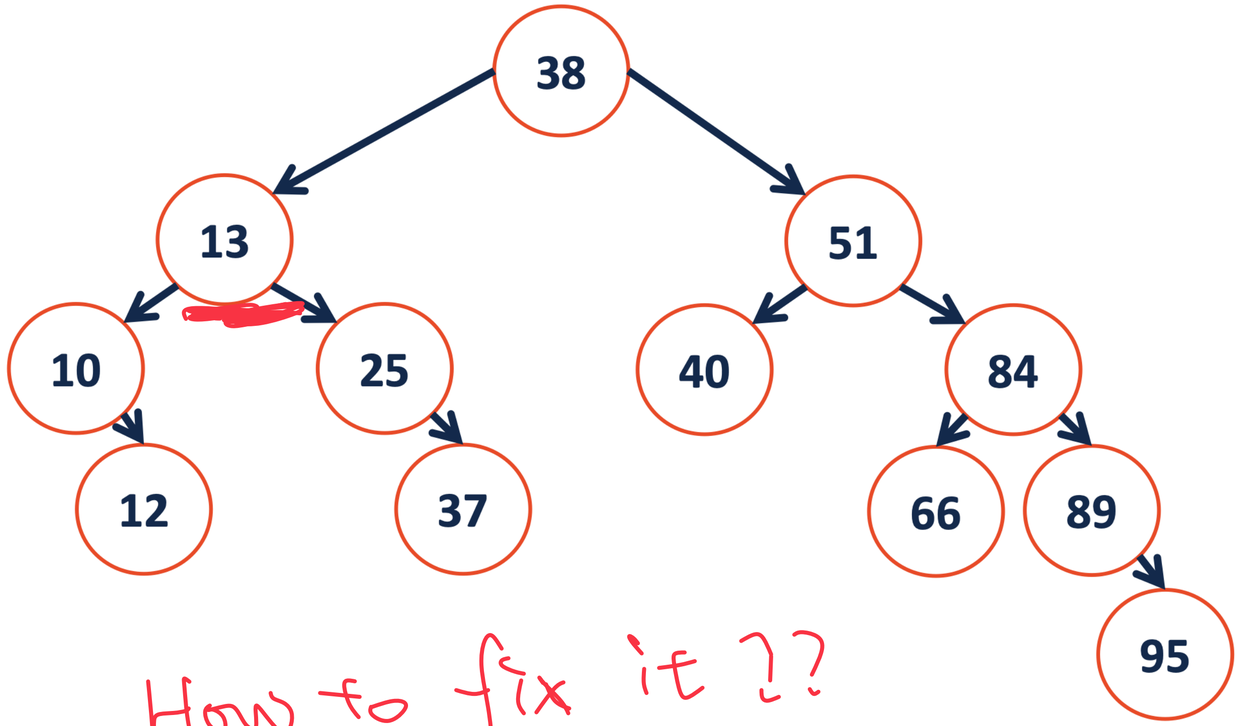
remove like in a linked list.

```
remove(25);
```



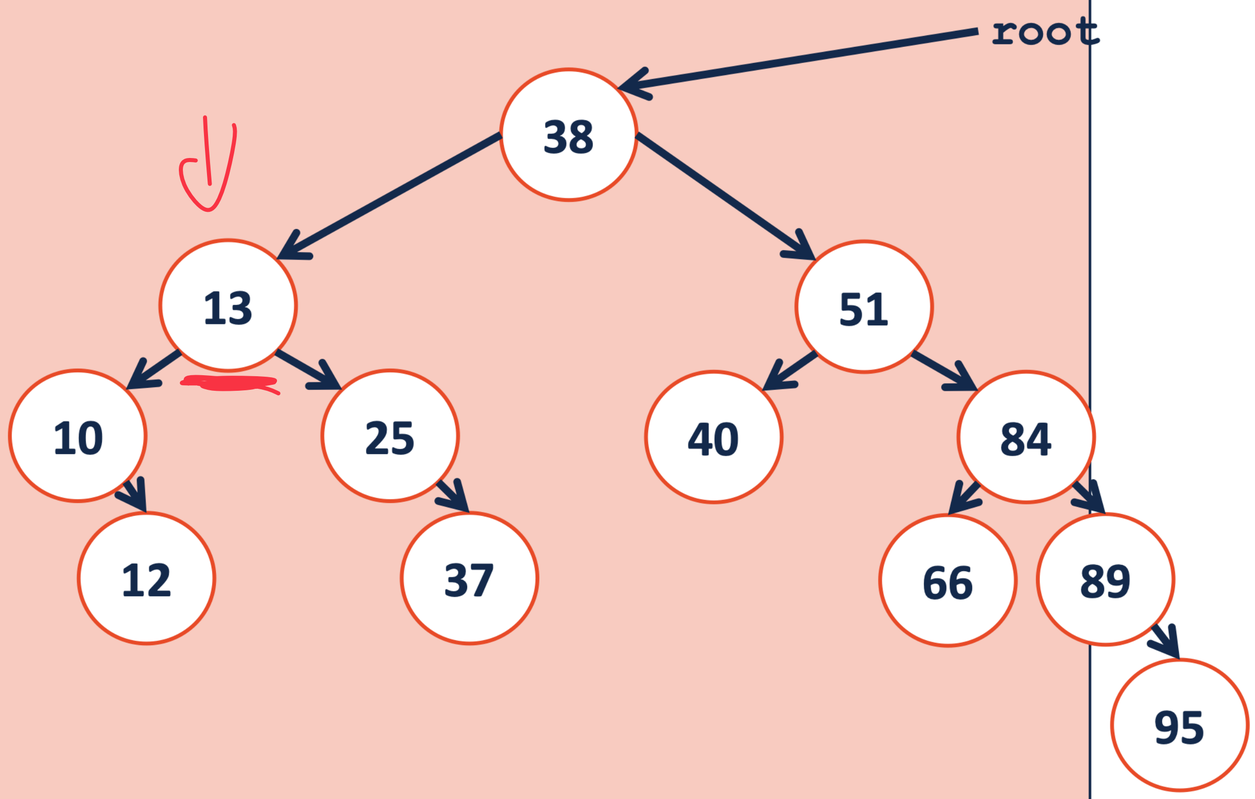
remove in linked list

```
remove(10);
```




```
remove(13);
```

```
1  template<class K, class V>
2  _____________________ remove(TreeNode *& root, const K & key) {
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26 }
```



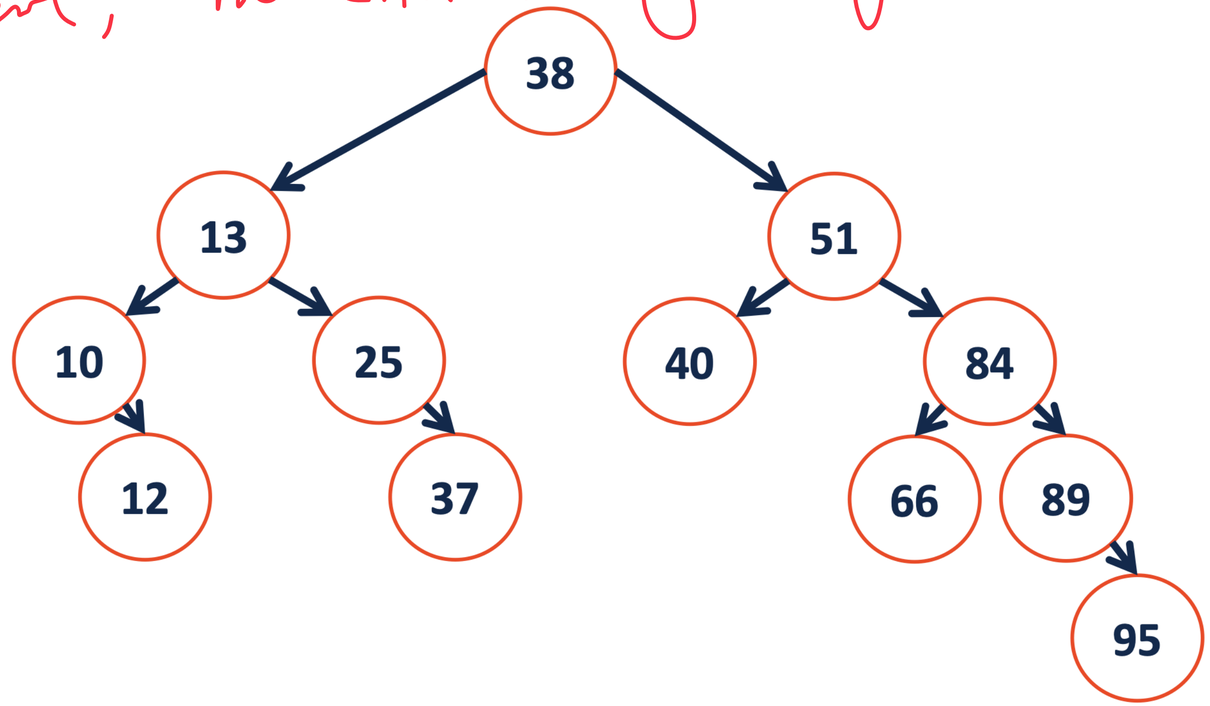
CS 225

Data Structures

Volodymyr Kindratenko

Slides courtesy of Wade Fagen-Ulmschneider

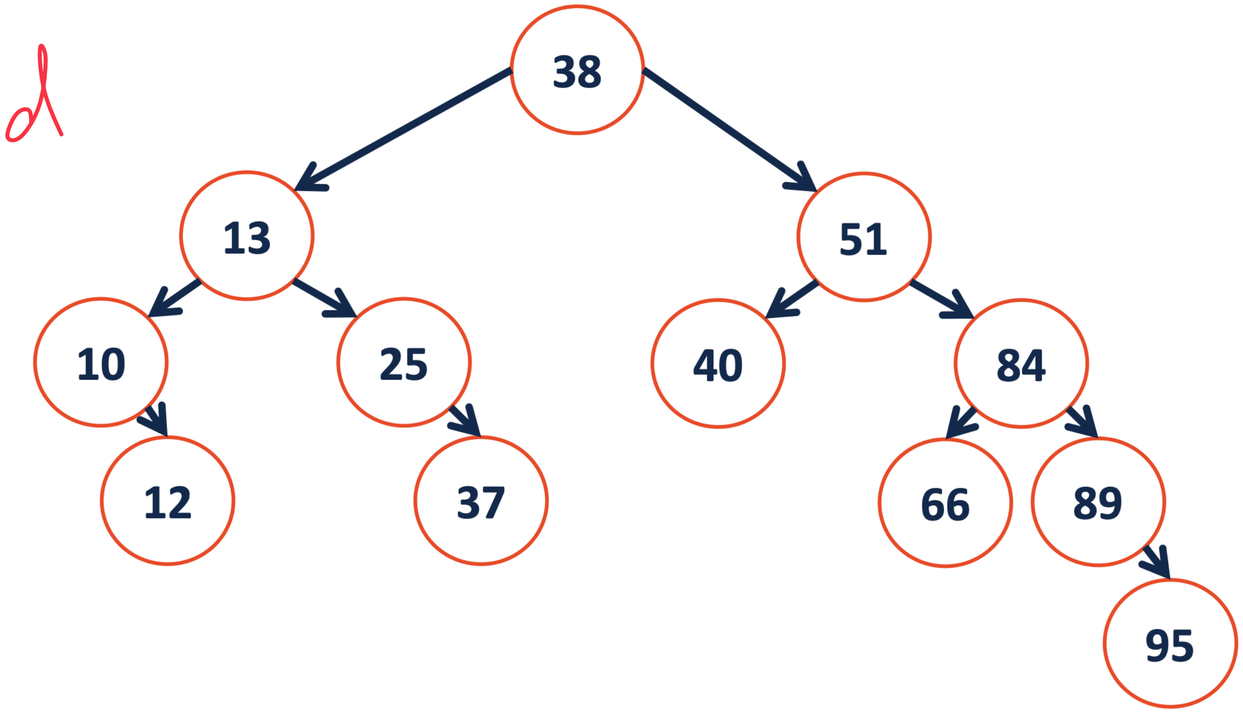
- leaf element, no child, very easy




```
remove(40);
```

- Remove in a linked list

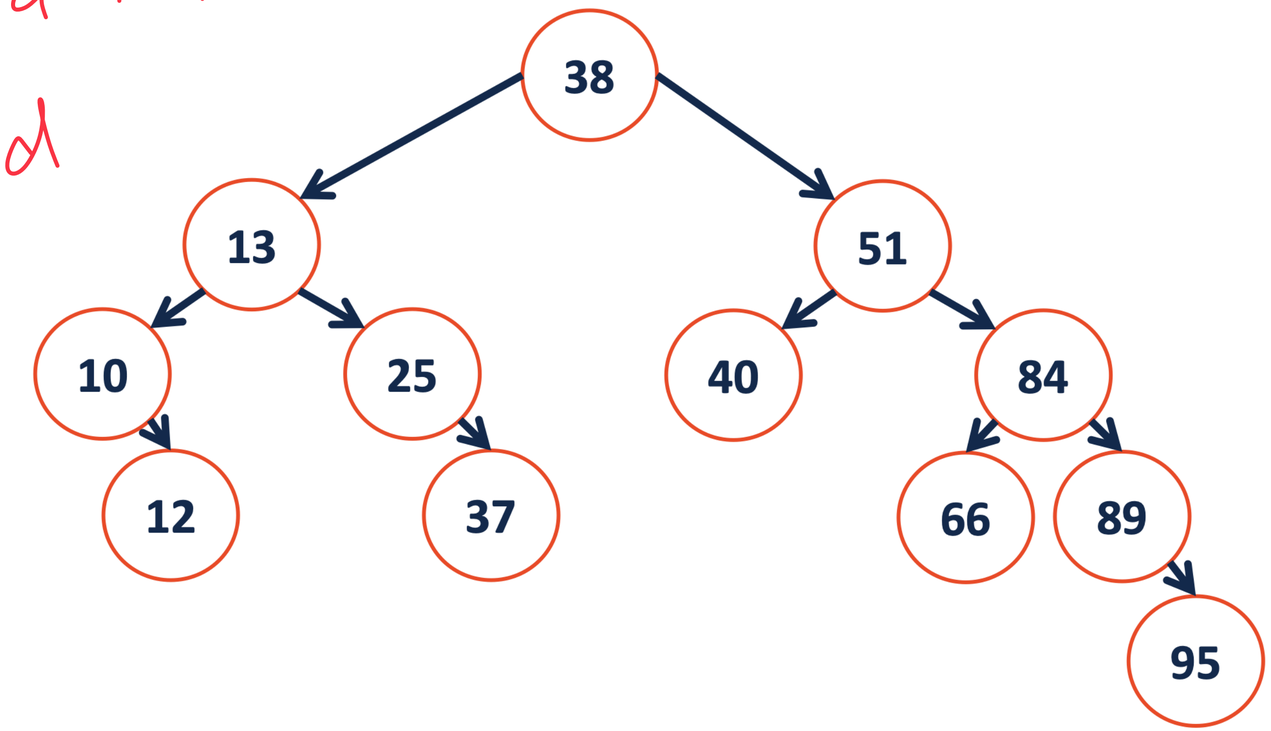
- one child




```
remove(25);
```

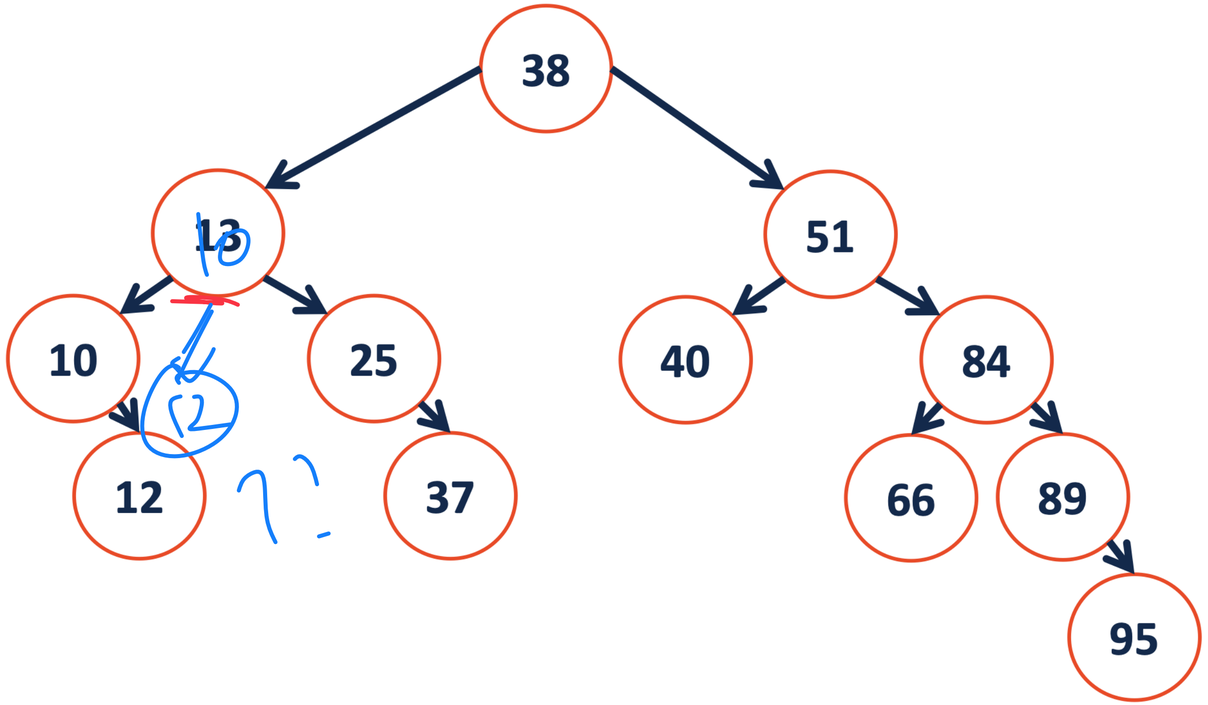
- Remove in a linked list

- One child



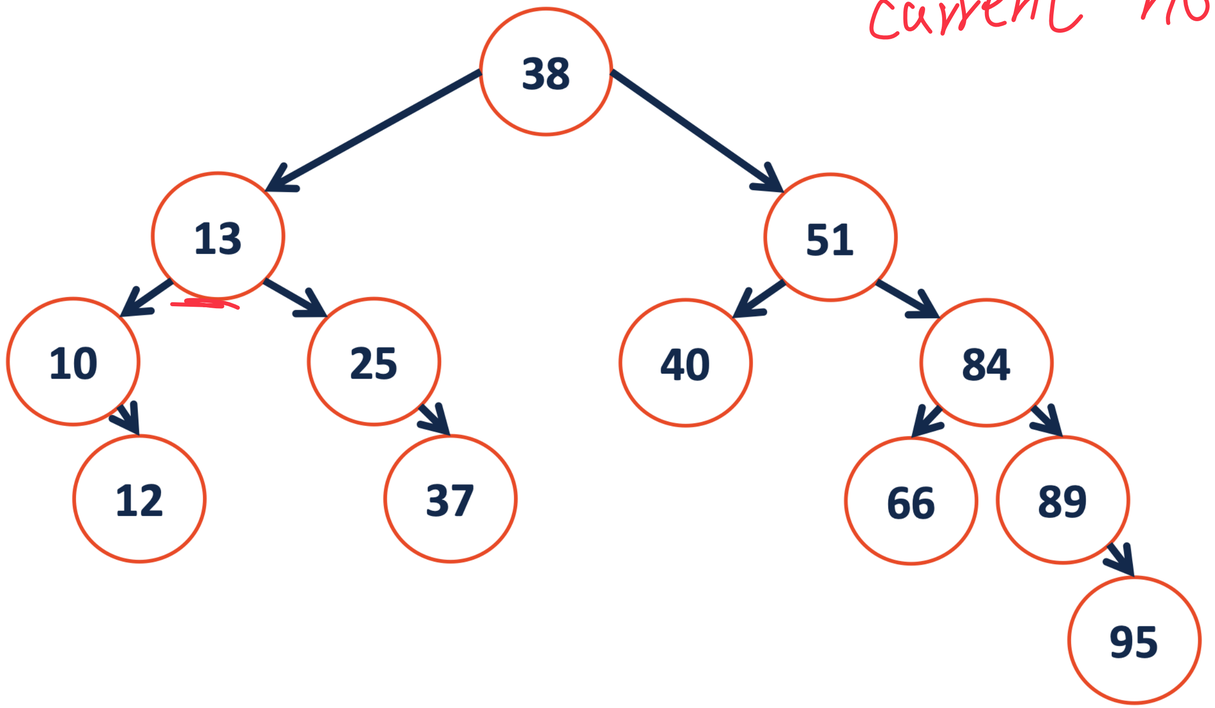

```
remove(10);
```

- How to arrange the children??




```
remove(13);
```

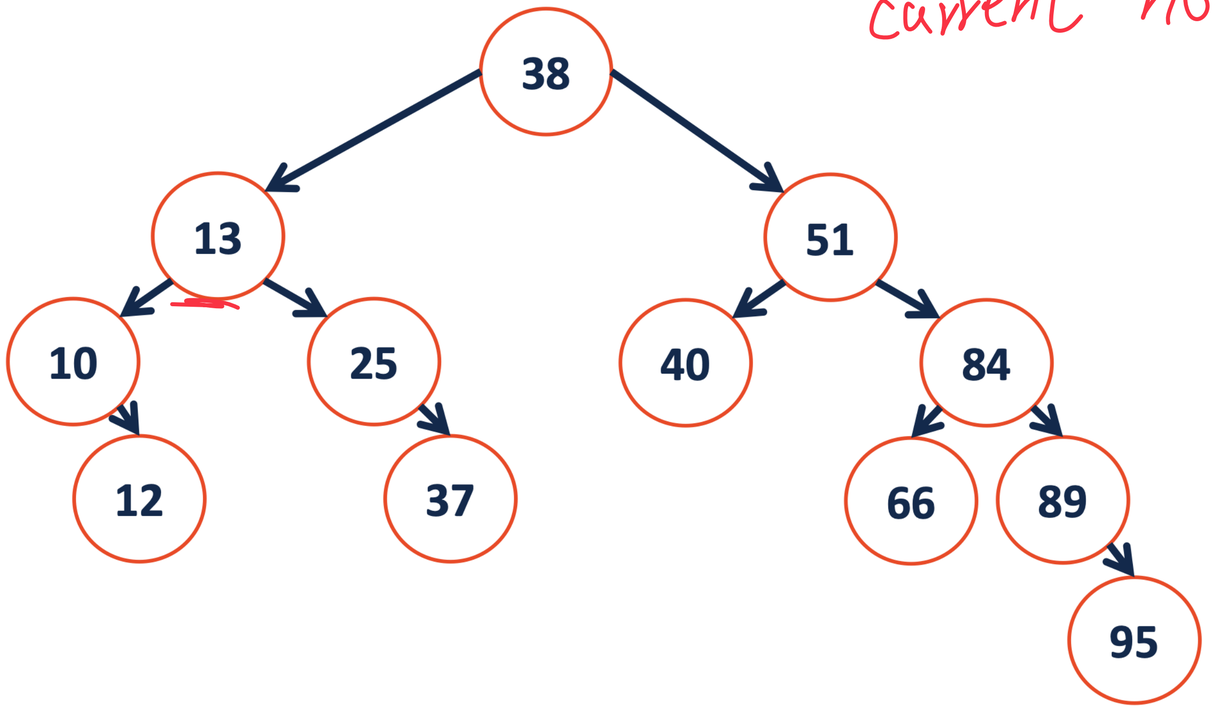
- How to arrange the children??
- In-order-predecessor(IOP): the in-order node directly before the current node




```
remove(13);
```

In-order: 10, 12, 13, 25, 37, 38, ...

- How to arrange the children??
- In-order-predecessor(IOP): the in-order node directly before the current node



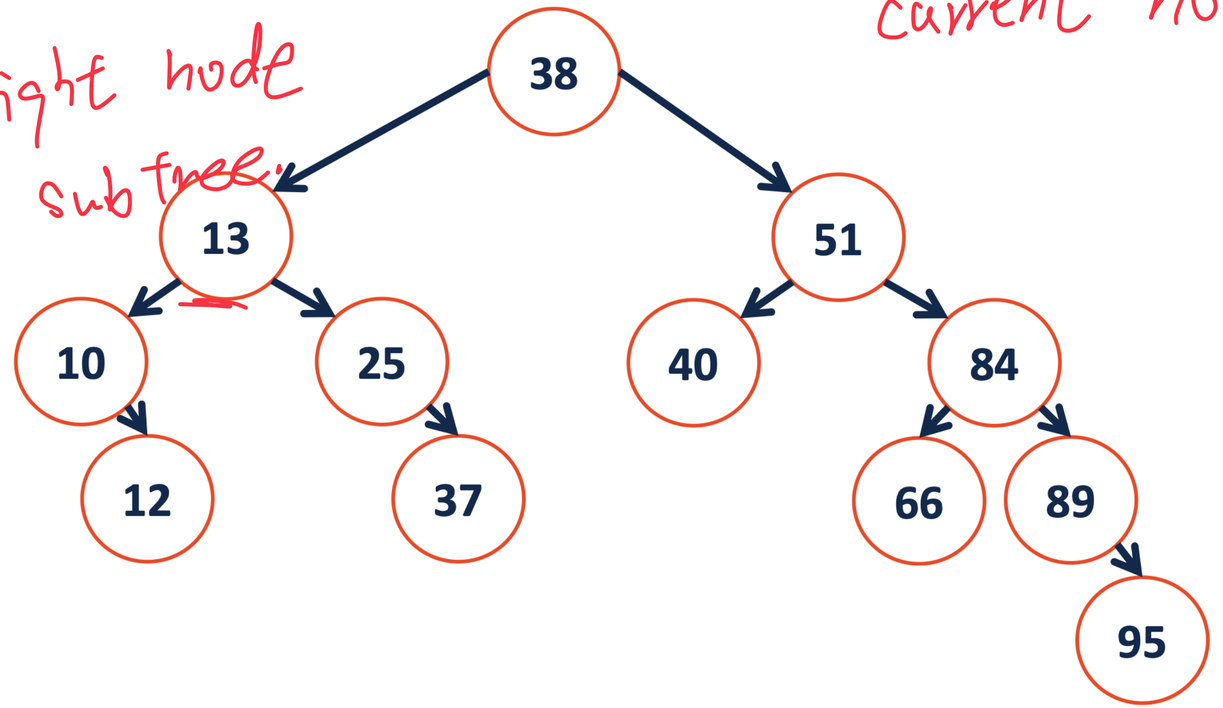
#38: IOP: 37

#51: IOP: 40

```
remove(13);
```

In-order: 10, 12, 13, 25, 37, 38, ...

- How to arrange the children??
- In-order-predecessor(IOP): the in-order node directly before the current node
- IOP: far right node in the left subtree



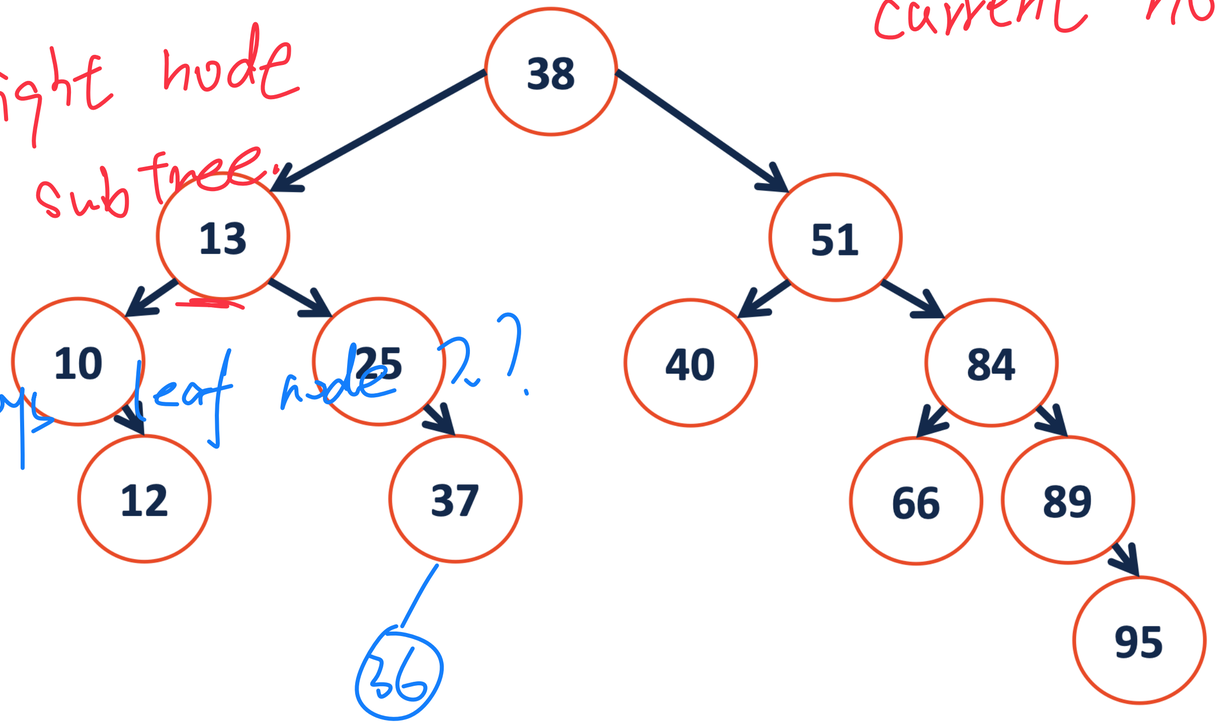
#38: IOP: 37

#51: IOP: 40

```
remove(13);
```

In-order: 10, 12, 13, 25, 37, 38, ...

- How to arrange the children??
- In-order-predecessor(IOP): the in-order node directly before the current node
- IOP: far right node in the left subtree.
- IOP is always leaf node??



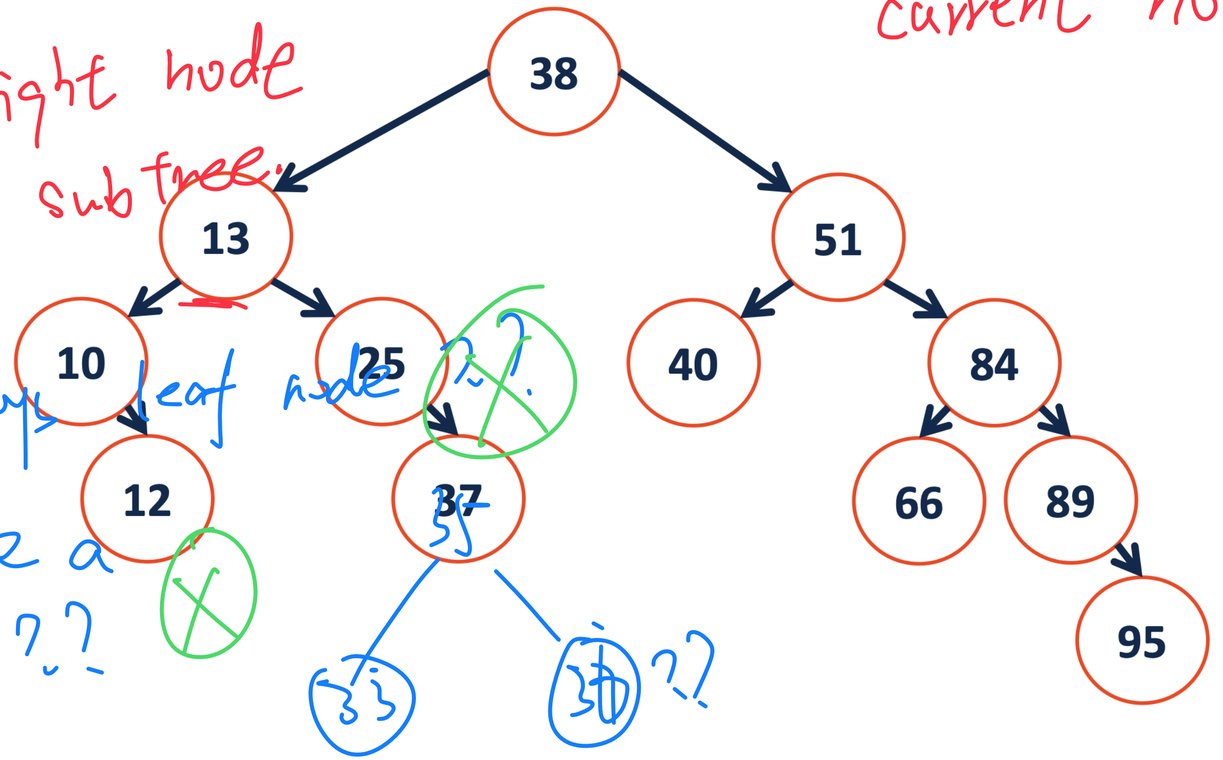
#38: IOP: 37

#51: IOP: 40

```
remove(13);
```

In-order: 10, 12, 13, 25, 37, 38, ...

- How to arrange the children??
- In-order-predecessor(IOP): the in-order node directly before the current node
- IOP: far right node in the left subtree.
- IOP is always leaf node
- IOP can have a right child ??



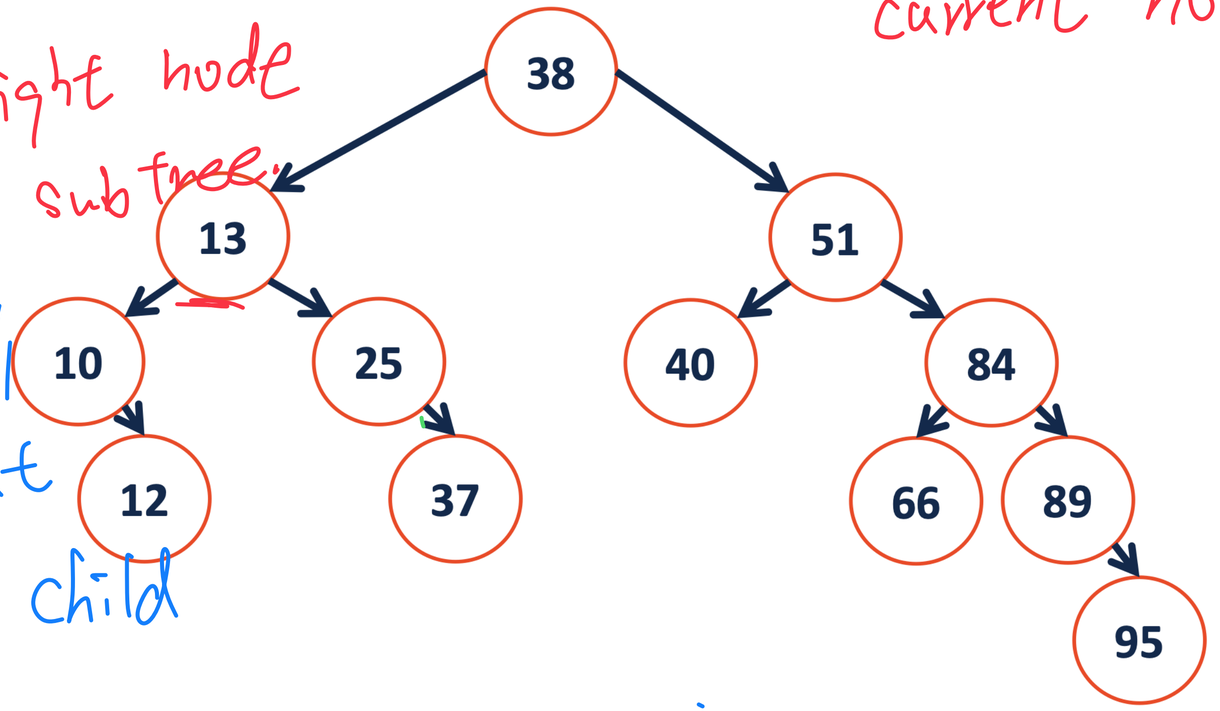
#38: IOP: 37

#51: IOP: 40

```
remove(13);
```

In-order: 10, 12, 13, 25, 37, 38, ...

- How to arrange the children??
- In-order-predecessor(IOP): the in-order node directly before the current node
- IOP: far right node in the left subtree.
- IOP has 0/1 children node, but never has right child



#38: IOP: 37

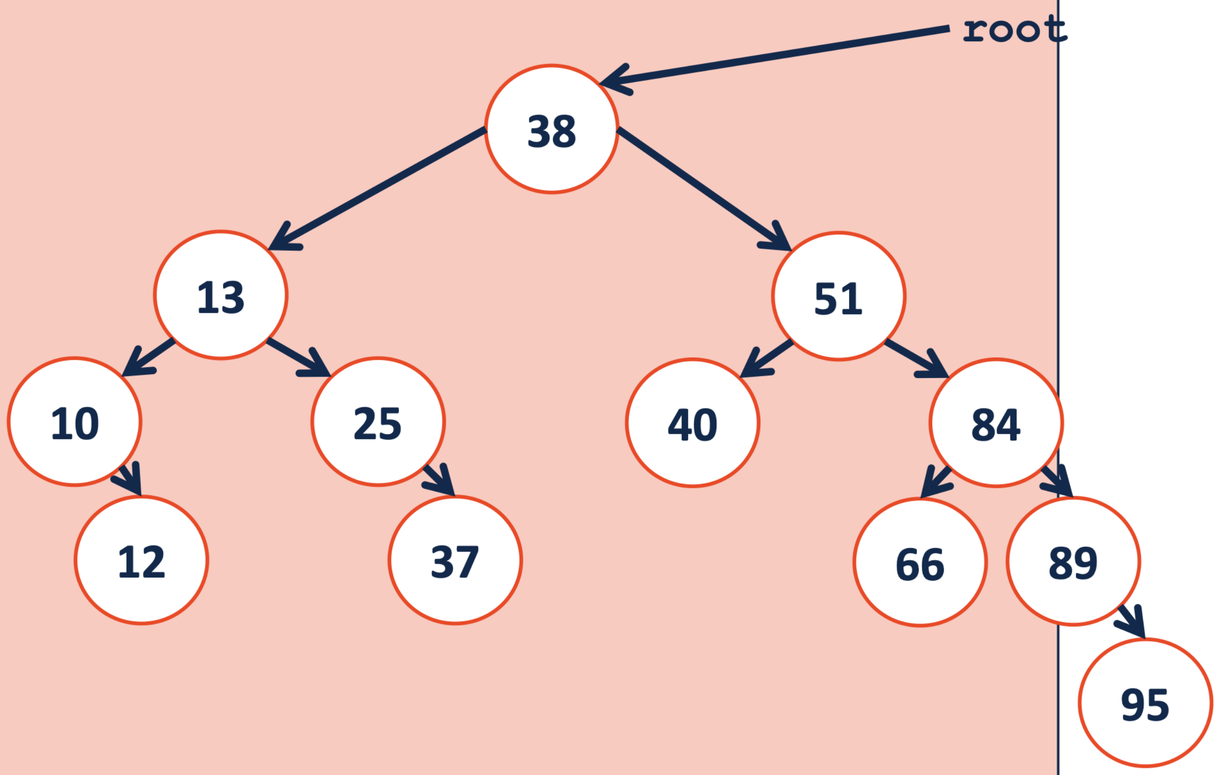
#51: IOP: 40

```
remove(13);
```

In-order: 10, 12, 13, 25, 37, 38, ...

```
1  template<class K, class V>
2  _____ remove(TreeNode *& root, const K & key) {
3
4    // case 1: 0 child
5
6
7
8    // case 2: 1 child
9
10
11
12
13   // case 3: 2 children
14
15
16
17
18
19
20
21
22
23
24
25
26 }
```

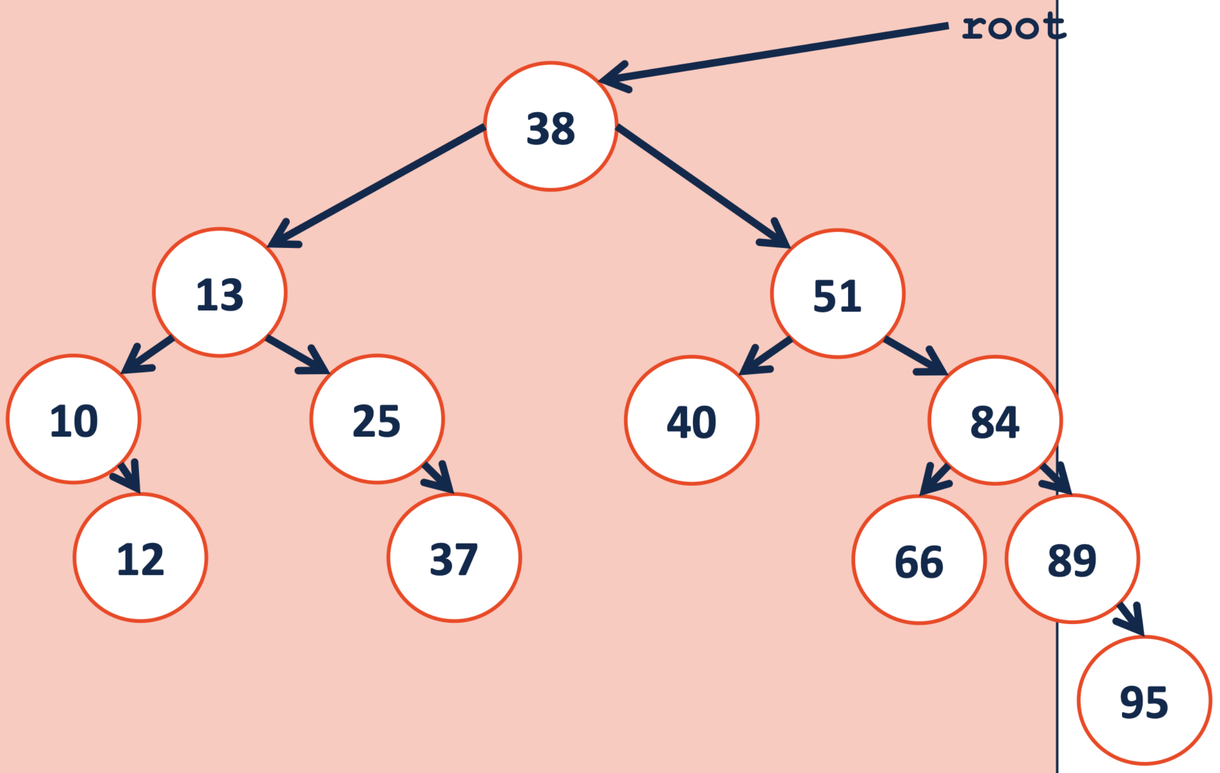
- rearrange with 2 ops, : Two children



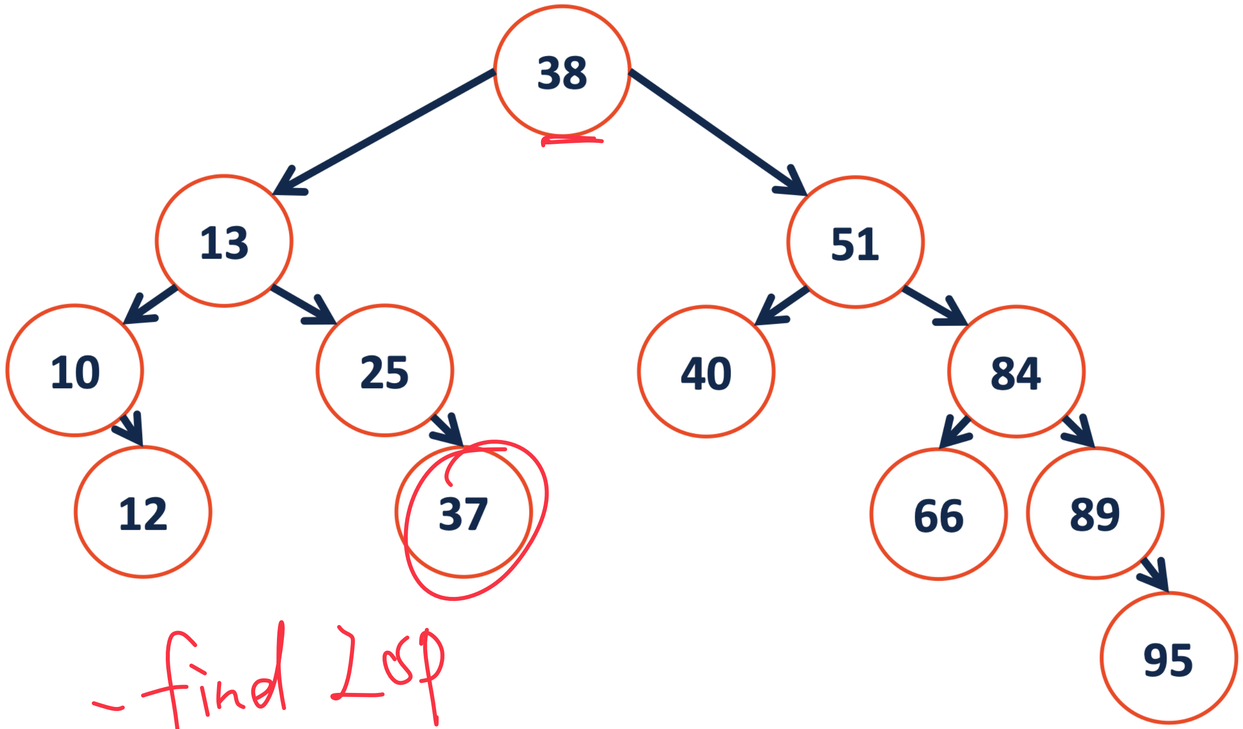
```
1  template<class K, class V>
2  _____________________________ remove(TreeNode *& root, const K & key) {
3
4
5
6
7
8
9
10
11
12 // case 3 : 2 children
13
14
15
16
17
18
19
20
21
22
23
24
25
26 }
```

- rearrange with IOPs, : Two children

① Find IOP

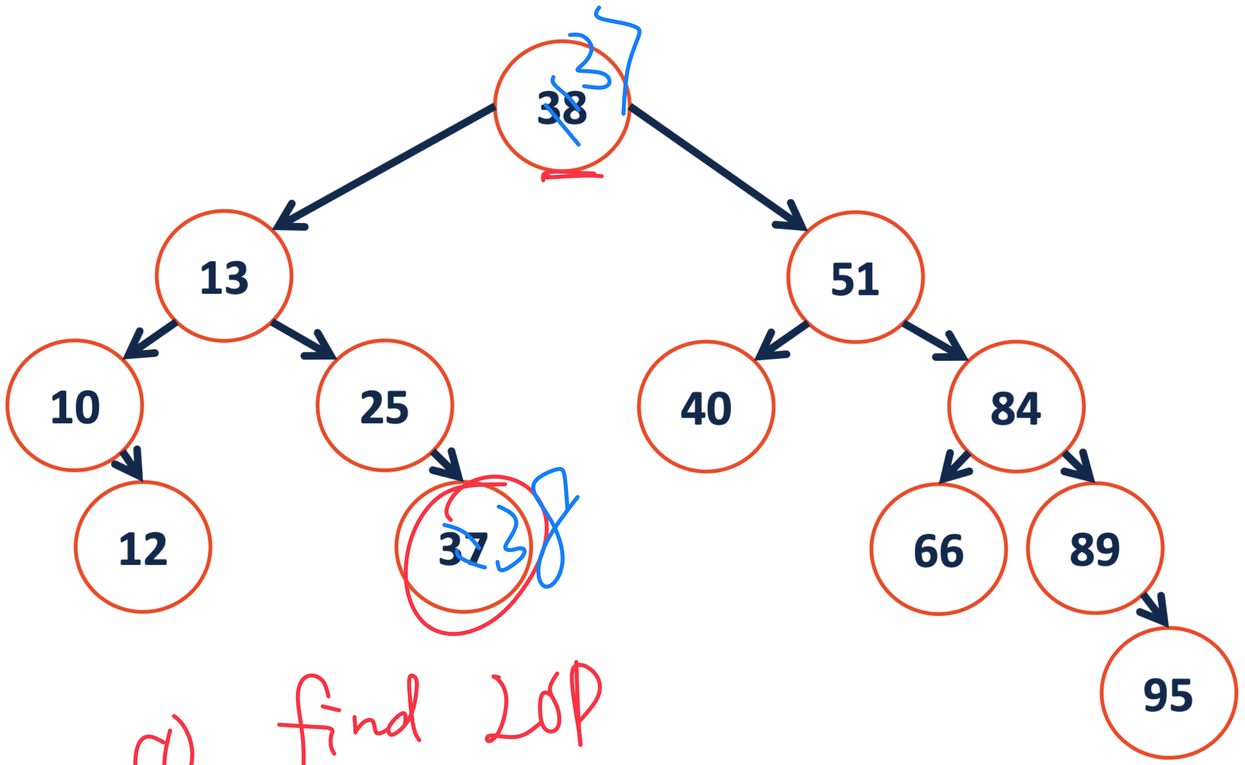


_remove(38)



① _find Top

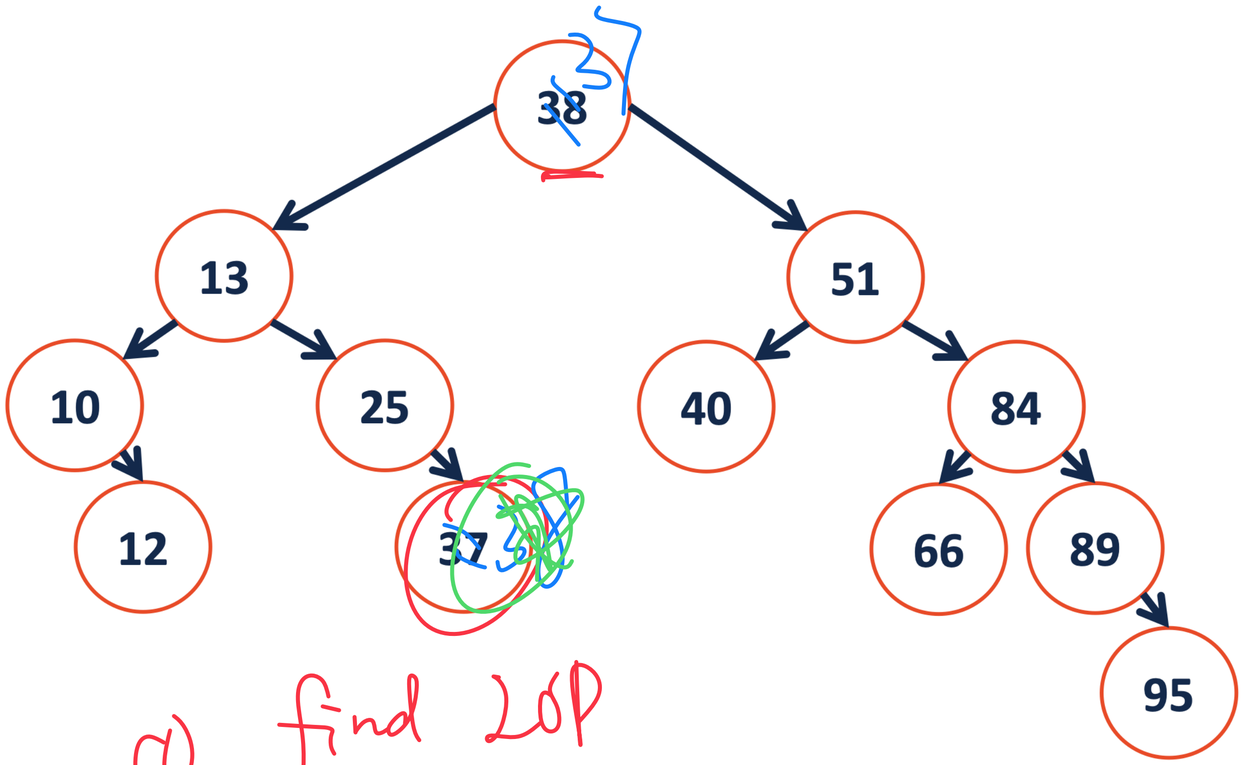
_remove(38)



① find IOP

② Swap current removal with IOP

_remove(38)

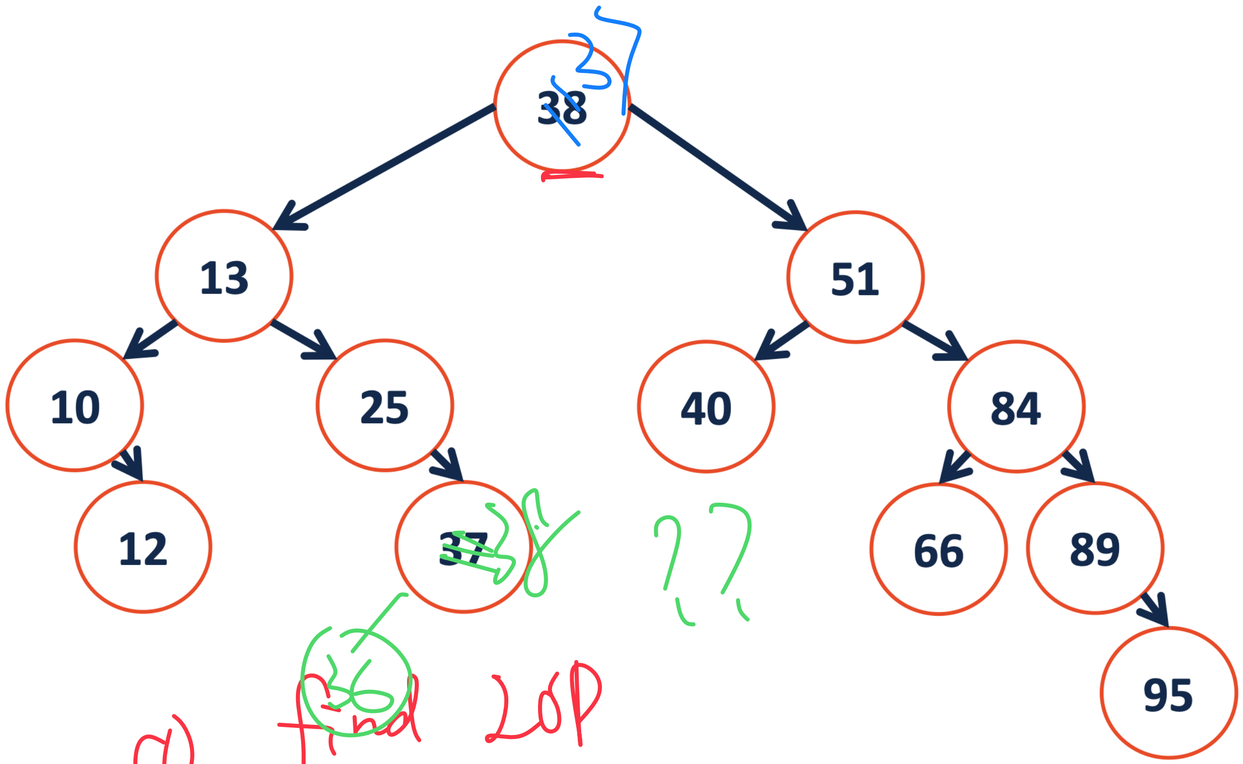


① find IOP

② Swap current removal with IOP

③ Delete removal

_remove(38)



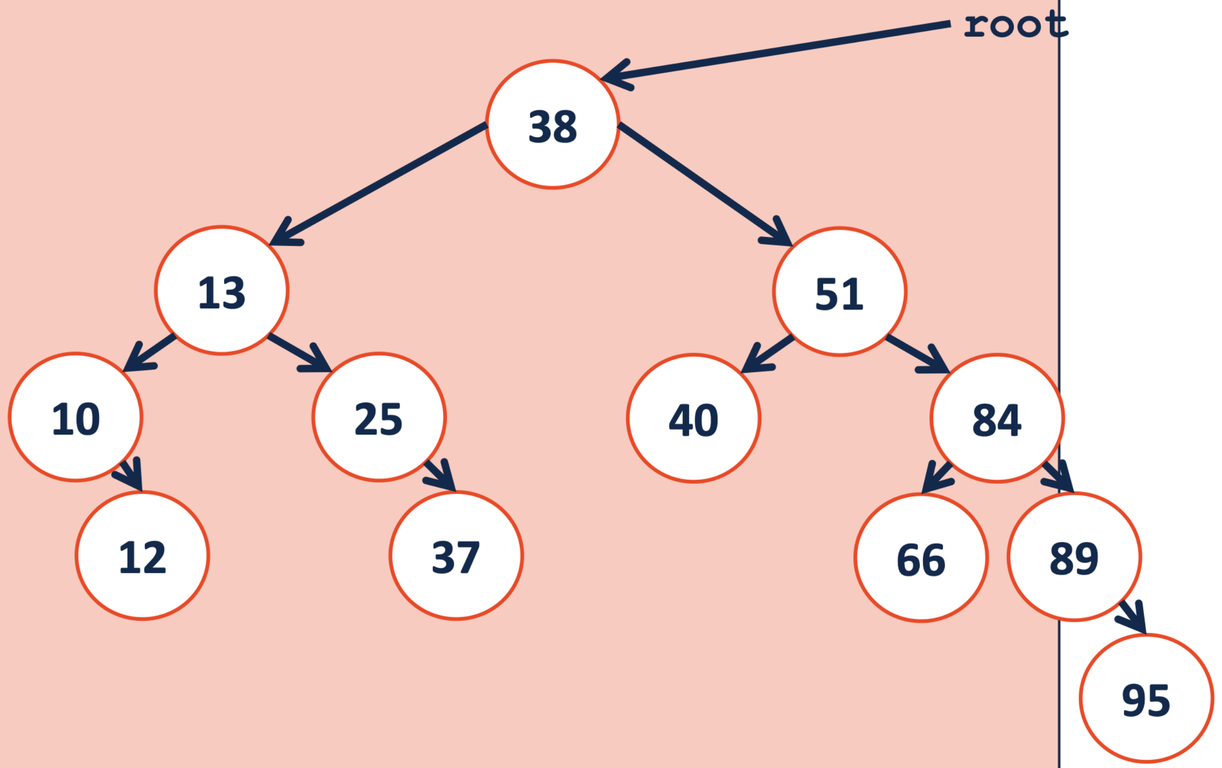
① find 2OP

② Swap current removal with 2OP

③ remove again → 0/1 child

```
1  template<class K, class V>
2  _____________________ remove(TreeNode *& root, const K & key) {
3  // case 1: 0 child
4
5
6
7  // case 2: 1 child
8
9
10
11
12 // case 3: 2 children
13
14
15   ① Find IOP
16
17
18   ② Swap
19
20
21
22   ③ _remove again
23
24
25
26 }
```

- rearrange with IOPs, : Two children

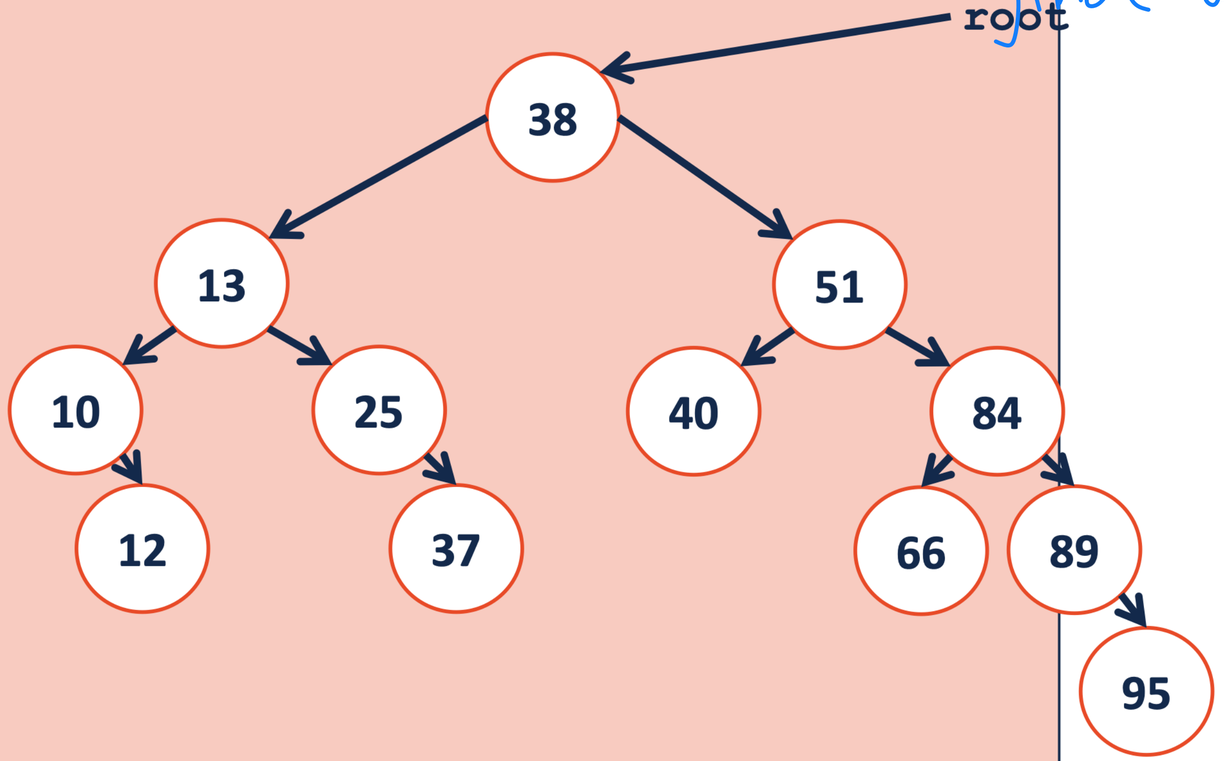


```
1  template<class K, class V>
2  _____ remove(TreeNode *& root, const K & key) {
3  // case 1: 0 child
4
5
6
7  // case 2: 1 child
8
9
10
11
12 // case 3: 2 children
13
14
15   ① Find IOP
16
17
18   ② Swap
19
20
21
22   ③ _remove again
23
24
25
26 }
```

- rearrange with IOPs, : Two children

running time :

O(h) ⇒ find node/ find IOPs



BST Analysis

Every operation that we have studied on a BST depends on the height of the tree: **O(h)**.

… what is this in terms of **n**, the amount of data?

We need a relationship between **h** and **n**:

h ≥ f(n) max element for a given height (need h+1 if we want to insert more)

h ≤ g(n) min element for a given height (will reduce to h+1 if we have less elements.

BST Analysis

Q: What is the maximum number of nodes in a tree of height **h**?

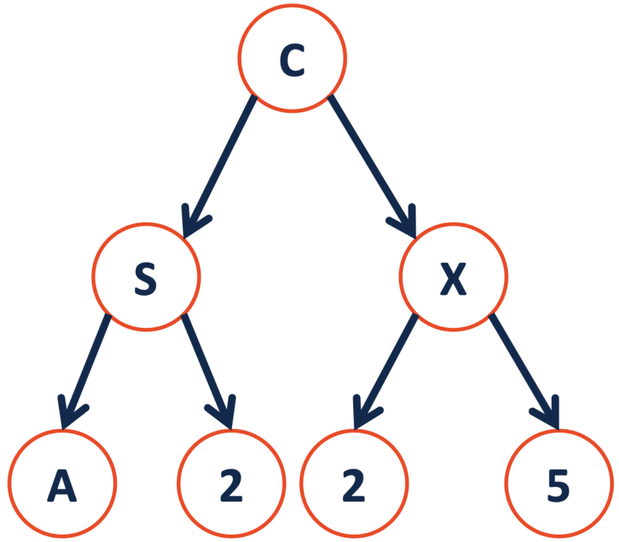
$T = \{\}$, $h = -1$, $n = 0$

$T = \{r, T_L, T_R\}$, $f^{-1}(h) \geq n$

$m(h)$: max nodes at a given height h

$m(-1) = 0$

$=$



BST Analysis

Q: What is the maximum number of nodes in a tree of height **h**?

$T = \{\}$, $h = -1$, $n = 0$

$T = \{r, T_L, T_R\}$, $f^{-1}(h) \geq n$

$m(h)$: max nodes at a given height h

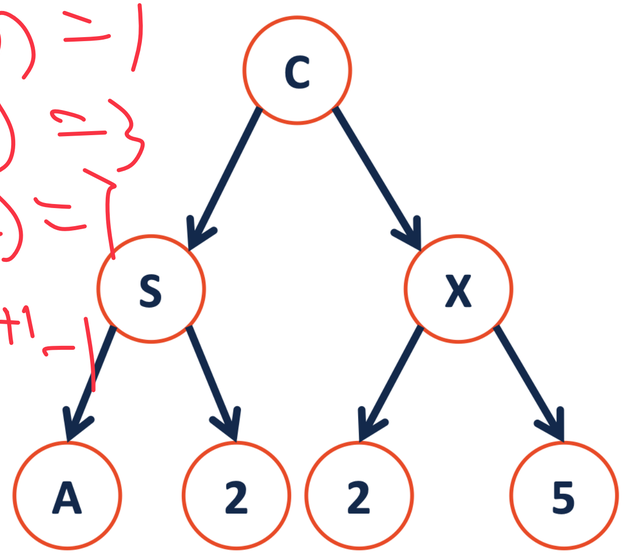
$m(h) = 2m(h-1) + 1 = 2^{h+1} - 1$

$m(-1) = 0$

$m(0) = 1$

$m(1) = 3$

$m(2) = 7$



BST Analysis

Q: What is the maximum number of nodes in a tree of height **h**?

$T = \{\}$, $h = -1$, $n = 0$

$T = \{r, T_L, T_R\}$, $f^{-1}(h) \geq n$

$m(h)$: max nodes at a given height h

$m(-1) = 0$

$m(0) = 1$

$m(1) = 3$

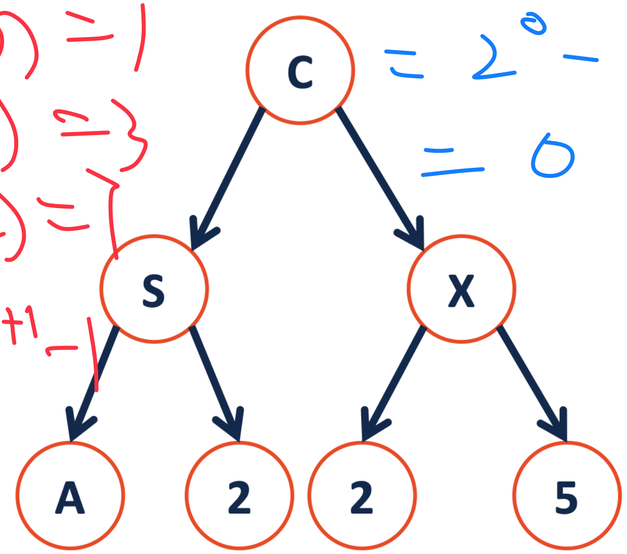
$m(2) = 7$

$m(h) = 2m(h-1) + 1 = 2^{h+1} - 1$

$m(-1) = 2^{-1+1} - 1$

$= 2^0 - 1$

$= 0$ ✓



BST Analysis

Q: What is the maximum number of nodes in a tree of height **h**?

$T = \{\}$, $h = -1$, $n = 0$

$T = \{r, T_L, T_R\}$, $f^{-1}(h) \geq n$

$m(h)$: max nodes at a given height h

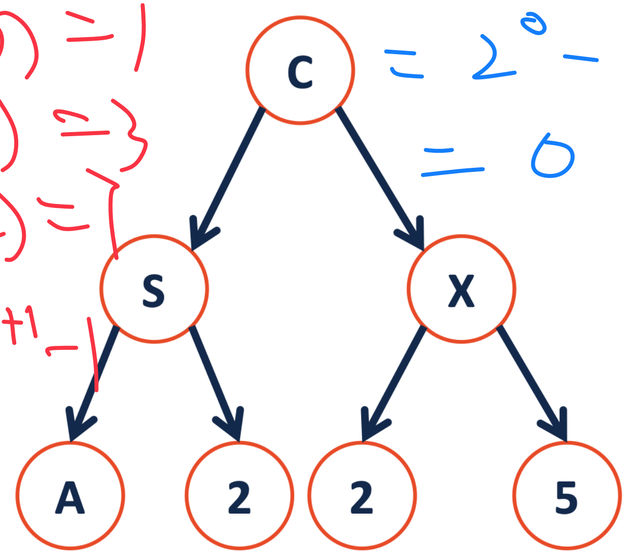
$m(-1) = 0$ $\quad m(-1) = 2^{-1+1} - 1$

$m(0) = 1$ $\quad = 2^0 - 1$

$m(1) = 3$ $\quad = 0$ ✓

$m(2) = 7$

$m(h) = 2m(h-1) + 1 = 2^{h+1} - 1$



$m(h) = 2m(h-1) + 1$

$= 2(2^{h-1+1} - 1) + 1$

$= 2 \cdot 2^h - 2 + 1$

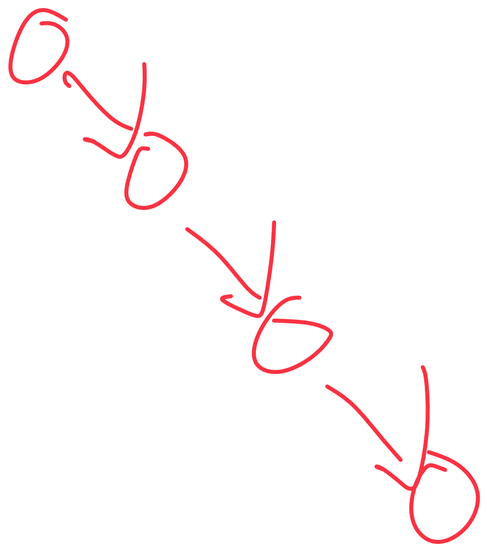
$= 2^{h+1} - 1$ ✓

proof by induction.

- other solution ??

BST Analysis

Q: What is the greatest possible height for a tree of **n** nodes?

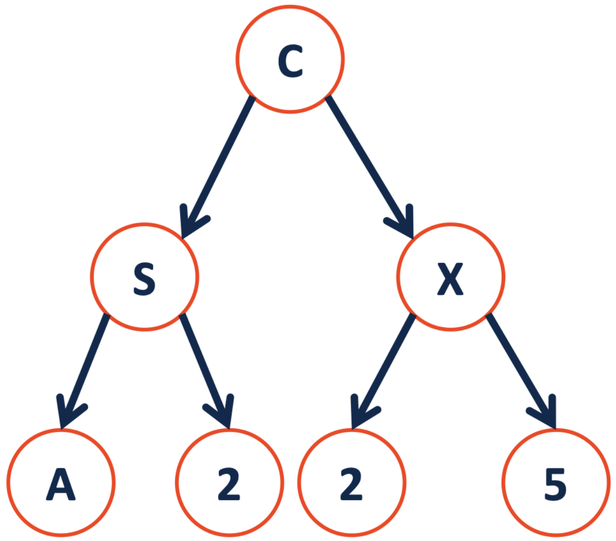


$n = 4$

$h = 3$

$h \leq n-1$

$O(n)$



BST Analysis

Therefore, for all BST:

Lower bound:

$n \geq M(h) \geq 2^{h+1} - 1 \quad \Rightarrow \quad h \sim O(\log n)$

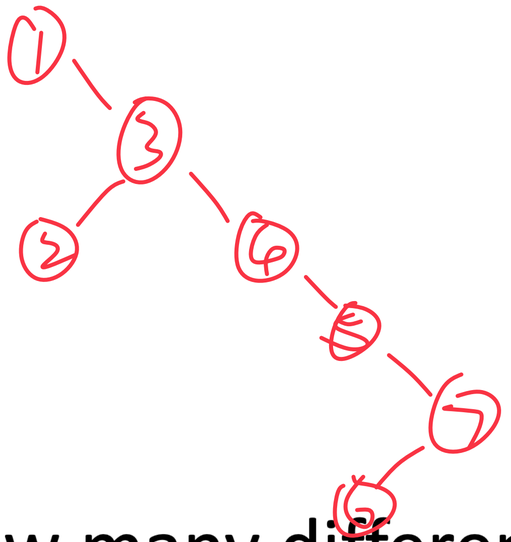
Upper bound:

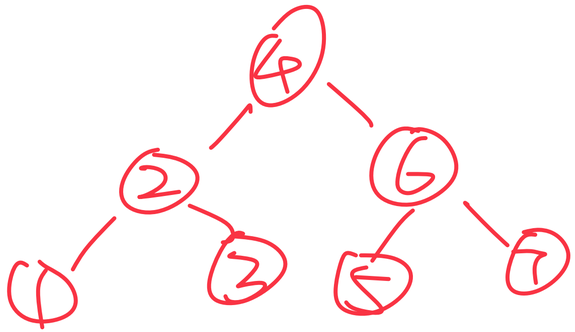
$O(n)$

BST Analysis

The height of a BST depends on the order in which the data is inserted into it.

ex: 1 3 2 4 5 7 6 **vs.** **4 2 3 6 7 1 5**





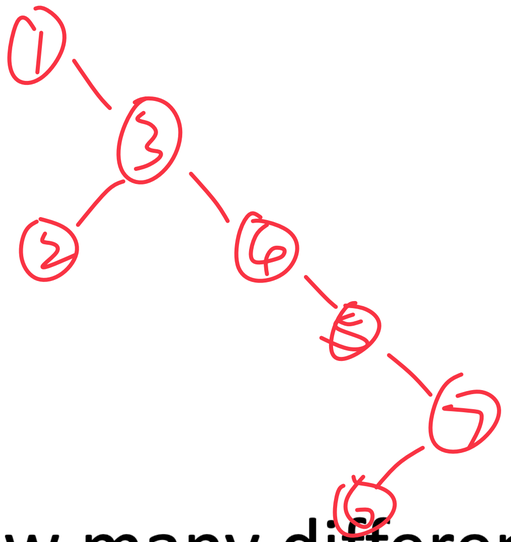
Q: How many different ways are there to insert keys into a BST?

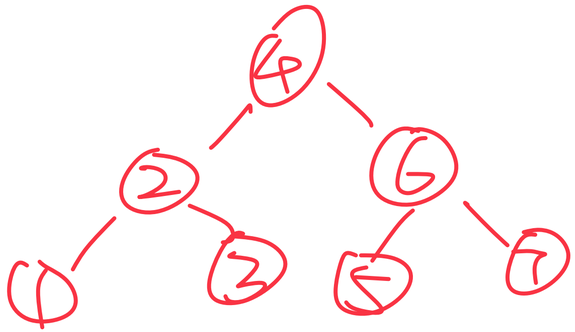
Q: What is the average height of all the arrangements?

BST Analysis

The height of a BST depends on the order in which the data is inserted into it.

ex: 1 3 2 4 5 7 6 **vs.** **4 2 3 6 7 1 5**





Q: How many different ways are there to insert keys into a BST?

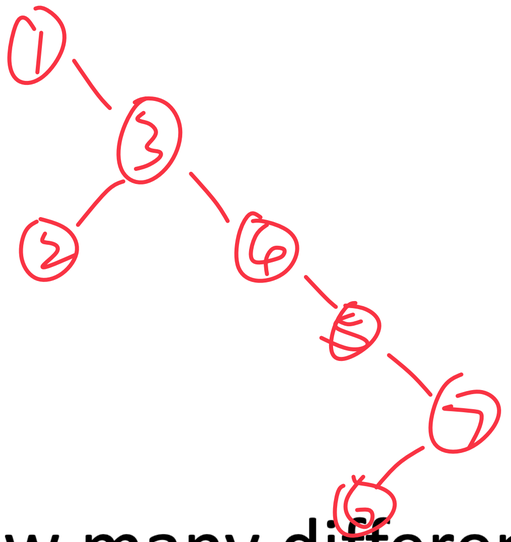
$O(n!)$

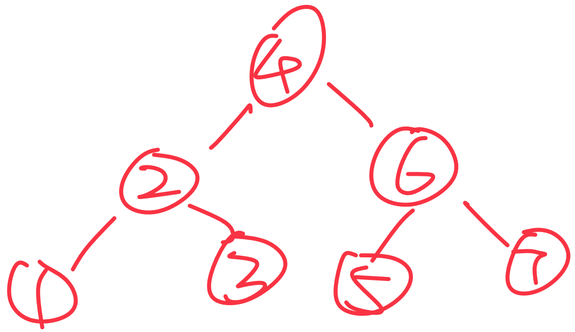
Q: What is the average height of all the arrangements?

BST Analysis

The height of a BST depends on the order in which the data is inserted into it.

ex: 1 3 2 4 5 7 6 **vs.** **4 2 3 6 7 1 5**





Q: How many different ways are there to insert keys into a BST?

$O(n!)$

Q: What is the average height of all the arrangements?

$O(\lg n)$

$E[X_n] \le 3 \lg n + O(1)$

randomized Algo.

BST Analysis – Running Time

Operation	BST Average Case	BST Worst Case	Sorted Array	Sorted List
find	O(lg n)			
insert	O(lg n)			
delete	O(lg n)			
traverse	O(n)			

BST Analysis – Running Time

Operation	BST Average Case	BST Worst Case	Sorted Array	Sorted List
find	O(lg n)	O(n)	O(lg n)	O(n)
insert	O(lg n)	O(n)	O(n)	Given location: O(1) Arbitrary: O(n)
delete	O(lg n)	O(n)	O(n)	⇓ the same.
traverse	O(n)	→	→	→